PRIVACY PRESERVING RANKED MULTI KEYWORD SEARCH FOR MULTIPLE DATA OWNERS IN CLOUD COMPUTING

A Project Report submitted to

JNTUA, Anantapuramu

In partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

Computer Science and Engineering

Submitted by

BATCH NO:22PR055

G. Naga Punith(22KB1A0548)

J.VenkataKishore(22KB1A0556)

K.Raghavendra(22KB1A0584)

N.Sasikanth(22KB1A05B1)

Under the esteemed Guidance of

Mrs. P. SUNEETHA B.Tech., M.Tech.,(Ph.D)

Assistant Professor, Dept. of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

N.B.K.R. INSTITUTE OF SCIENCE AND TECHNOLOGY

(Autonomous)

VIDYANAGAR, TIRUPATI DIST, AP - 524413

April-2026

Website:www.nbkrist.org. Ph: 08624-228 247
Email: ist@nbkrist.org. Fax: 08624-228 257

N.B.K.R. INSTITUTE OF SCIENCE & TECHNOLOGY
(Autonomous)

(Approved by AICTE: Accredited by NBA: Affiliated to JNTUA,Anantapuramu)

An ISO 9001-2000 Certified Institution

Vidyanagar -524 413, Tirupati District, Andhra Pradesh, India

---

BONAFIDE CERTIFICATE

This is to certify that the project work entitled "PRIVACY PRESERVING RANKED MULTI KEYWORD SEARCH FOR MULTIPLE DATA OWNERS IN CLOUD COMPUTING" is a bonafide work done by G. Naga Punith(22KB1A0548), J.VenkataKishore(22KB1A0556), K.Raghavendra(22KB1A0584),N.Sasikanth(22KB1A05B1) in the Department of Computer Science & Engineering,N.B.K.R. Institute of Science & Technology,Vidyanagar and is submitted to JNTUA, Anantapuramu in the partial fulfillment for the award of B.Tech degree in Computer Science & Engineering. This work has been carried out under my supervision.

Mrs. P. Suneetha	**Mr.M.Hari Babu**
Assistant Professor	**P.O.P & HOD**
Department of CSE	**Department of CSE**
NBKRIST, Vidyanagar	**NBKRIST, Vidyanagar**

Submitted for the Viva-Voce Examination held on_____

Examiner-1 Examiner-2

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of a project would be incomplete without the people who made it possible with their constant guidance and encouragement crowned our efforts with success.

We would like to express our profound sense of gratitude to our project guide **Mrs.P. Suneetha , Assistant Professor Department of Computer Science & Engineering, N.B.K.R.I.S.T (affiliated to JNTUA, Anantapuramu),** Vidyanagar, for his masterful guidance and the constant encouragement throughout the project. Our sincere appreciation for his suggestions and unmatched services without, which this work would have been an unfulfilled dream.

We convey our special thanks to **Dr. Y. VENKATA RAMI REDDY respectable chairman of N.B.K.R. Institute of Science and Technology,** for providing excellent infrastructure in our campus to complete the project.

We convey our special thanks to **Sri N. RAM KUMAR respectable correspondent of N.B.K.R. Institute of Science and Technology,** for providing excellent infrastructure in our campus to complete the project.

We are grateful to **Dr. V. Vijaya Kumar Reddy, Director, N.B.K.R Institute of Science and Technology** for allowing us to avail all the facilities in the college.

We express our sincere gratitude to **Dr. M. HariBabu, Professor of Practice(P.O.P), Head of Department, Computer Science & Engineering,** for providing exceptional facilities for the successful completion of our project work.

We would like to convey our heart full thanks to **Staff members, Lab technicians, and our friends,** who extended their cooperation in making this project a successful one.

We would like to thank one and all who have helped us directly and indirectly to complete this project successfully

ABSTRACT

Cloud service models intrinsically cater to multiple tenants. In current multi-tenancy model, cloud service providers isolate data within a single tenant boundary with no or minimum cross-tenant interaction. With the booming of cloud applications, allowing a user to search across tenants is crucial to utilize stored data more effectively. However, conducting such a search operation is inherently risky, primarily due to privacy concerns. Moreover, existing schemes typically focus on a single tenant and are not well suited to extend support to a multi-tenancy cloud, where each tenant operates independently. In this article, to address the above issue, we provide a privacy- preserving, verifiable, accountable, and parallelizable solution for "privacy-preserving keyword search problem" among multiple independent data owners. We consider a scenario in which each tenant is a data owner and a user's goal is to efficiently search for granted documents that contain the target keyword among all the data owners. We first propose a verifiable yet accountable keyword searchable encryption (VAKSE) scheme through symmetric bilinear mapping. For verifiability, a message authentication code (MAC) is computed for each associated piece of data. To maintain a consistent size of MAC, the computed MACs undergo an exclusive OR operation. For accountability, we propose a keyword-based accountable token mechanism where the client's identity is seamlessly embedded without compromising privacy. Furthermore, we introduce the parallel VAKSE scheme, in which the inverted index is partitioned into small segments and all of them can be processed synchronously. We also conduct formal security analysis and comprehensive experiments to demonstrate the data privacy preservation and efficiency of the proposed schemes, respectively.

TABLE OF CONTENTS

CHAPTER NO	CHAPTER NAME	PAGE NO
CHAPTER 1:	**INTRODUCTION**	**1 – 5**
	1.1 Introduction	1
	1.2 Problem Statement	2
	1.3 Objectives And Scope	3 - 4
	1.4 Summary	4
CHAPTER 2:	**SURVEY OF THE LITERATURE**	**5 - 10**
	2.1 Introduction	5
	2.2 Literature Survey	6 - 9
	2.3 Existing System	9
	2.4 Proposed System	9 - 10
	2.5 Feasibility analysis	10
CHAPTER 3:	**DESIGN**	**11 - 17**
	3.1. System Design	11 - 15
	3.1.1. Description about Modules	16
	3.2. Detailed Design of the Project	17
CHAPTER 4 :	**IMPLEMENTATION**	**18 - 60**
	4.1 Requirements	18
	4.1.1 H/W requirements	18
	4.1.2 S/W requirements	18 - 55
	4.2 ScreenShots	56 - 60
CHAPTER 5:	**CONCLUSION AND FUTURE ENHANCEMENTS**	**61**
	5.1 Conclusion	61
	5.2 Future enhancements	61
CHAPTER 6:	**BIBILOGRAPHY**	**62**

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Cloud computing has emerged as one of the most transformative technologies in modern information systems, enabling organizations and individuals to store, manage, and access large volumes of data over the internet without maintaining local infrastructure. With the rapid growth of digital data across domains such as healthcare, education, finance, and e-commerce, cloud platforms have become essential for scalable and cost-effective data management. However, as data outsourcing becomes more common, concerns regarding data security and privacy have intensified. Sensitive information stored in the cloud is often vulnerable to unauthorized access, data leakage, and misuse, especially when cloud service providers are not fully trusted. To address these challenges, encryption techniques are widely adopted, ensuring that data remains protected even when stored on external servers. While encryption guarantees confidentiality, it also introduces a major limitation: performing efficient search operations on encrypted data becomes difficult.

Searchable encryption has been proposed as a solution to this problem, allowing users to search encrypted data without decrypting it. Traditional searchable encryption techniques enable keyword-based search, where users generate search tokens that allow the cloud server to locate relevant documents without revealing the actual content. Although these methods improve usability, most of the existing solutions are designed for single data owner environments. In such systems, a single entity controls the data, encryption keys, and access permissions. This assumption does not hold true in modern cloud environments, where multiple independent users or organizations (tenants) store their data on shared infrastructure. In multi-tenant systems, each data owner operates independently, with different security policies and no willingness to share encryption keys. This creates a significant challenge when a user needs to perform a search across multiple data owners while preserving privacy and security.

1.2 PROBLEM STATEMENT

Existing systems suffer from several limitations when extended to multi-owner scenarios. First, they lack scalability, as users are required to send separate search queries to each data owner, increasing computational overhead and latency. Second, many solutions rely on a shared master key among all data owners, which is impractical and poses serious security risks. Third, these systems do not provide fine-grained access control, meaning users either gain full access to data or no access at all, without restrictions based on specific keywords. Furthermore, most existing approaches assume that the cloud server behaves honestly, even though it may be curious to learn sensitive information. As a result, there is no mechanism to verify whether the search results returned by the server are correct and complete. Additionally, user accountability is not addressed, making it impossible to trace misuse of search tokens or unauthorized sharing of access rights.

To better understand the practical implications of these limitations, consider a real-world case study in the healthcare domain. In a cloud-based healthcare system, multiple hospitals act as independent data owners, each storing encrypted patient records in the cloud. A research organization or authorized medical practitioner may need to search across these distributed datasets to find relevant information, such as patients with specific symptoms or medical histories.

In existing systems, the researcher would need to query each hospital separately or rely on a centralized authority that manages all encryption keys. This approach is inefficient, time-consuming, and compromises data privacy. Moreover, if the cloud server returns incomplete or manipulated results, there is no mechanism to detect such issues. This can lead to incorrect medical analysis and potentially harmful decisions. Additionally, if a user shares their search credentials with unauthorized individuals, the system cannot track or prevent such misuse, leading to further security risks.

Another example can be observed in the financial sector, where multiple banks store encrypted transaction data on cloud servers. Regulatory authorities or auditors may need

to search across these datasets to detect fraudulent activities or compliance violations. In the absence of a secure multi-owner searchable encryption system, performing such operations becomes complex and inefficient. The lack of verification mechanisms can result in inaccurate reporting, while the absence of accountability increases the risk of insider threats. These real-world scenarios highlight the urgent need for a secure, efficient, and privacy-preserving solution that supports multi-keyword
search across multiple independent data owners.

1.3 OBJECTIVES AND SCOPE

The proposed system addresses these challenges by introducing a privacy-preserving ranked multi-keyword search framework designed specifically for multi-tenant cloud environments. The system is built upon a novel approach that combines searchable encryption with verification, accountability, and parallel processing capabilities. In this model, each data owner encrypts their data independently and uploads it to the cloud without sharing encryption keys with others. An inverted index is generated to map keywords to encrypted documents, enabling efficient search operations. When a user wants to search for specific keywords, a secure search token is generated that embeds the user's identity, ensuring accountability while maintaining privacy.

One of the key features of the proposed system is verifiability. Unlike existing solutions, this system ensures that users can verify the correctness and completeness of search results returned by the cloud server. This is achieved using message authentication codes, which provide a mechanism to detect any tampering or omission of data. As a result, users can trust the search results even when the cloud server is not fully trusted. Another important feature is accountability, which allows the system to track the origin of search tokens and identify users who misuse their access rights. This prevents unauthorized sharing of credentials and enhances overall system security.
The proposed system also introduces a parallel search mechanism to improve performance. In large-scale cloud environments, searching across massive datasets can be time-consuming. To address this, the system partitions the inverted index into smaller

segments that can be processed simultaneously. This parallelization significantly reduces search time and improves efficiency, making the system suitable for real-world applications with high data volumes. Additionally, the system supports fine-grained access control, allowing data owners to define specific permissions for different keywords. This ensures that users can only access authorized information, enhancing data privacy and security.

Compared to existing systems, the proposed solution offers several advantages. It eliminates the need for shared master keys, allowing data owners to maintain independence and control over their data. It enables efficient multi-keyword search across multiple data owners with a single query, reducing computational overhead. The inclusion of verification and accountability mechanisms ensures trustworthiness and security, addressing the limitations of traditional approaches. Furthermore, the use of parallel processing enhances scalability and performance, making the system practical for large-scale cloud environments.

1.4 SUMMARY

The increasing reliance on cloud computing for data storage and management has created a need for secure and efficient search mechanisms that preserve data privacy. Existing searchable encryption techniques are not sufficient to handle the complexities of multi-tenant environments, where multiple independent data owners coexist. The proposed system provides a comprehensive solution by integrating privacy preservation, verifiability, accountability, and performance optimization into a unified framework. Through real-world case studies in healthcare and finance, it is evident that such a system is essential for enabling secure data sharing and analysis in modern cloud-based applications.

This approach not only enhances data security but also improves usability and efficiency, paving the way for more reliable and trustworthy cloud computing systems.

CHAPTER-2

SURVEY OF THE LITERATURE

2.1 INTRODUCTION

Cloud computing has emerged as one of the most transformative technologies in modern information systems, enabling organizations and individuals to store, manage, and access large volumes of data over the internet without maintaining local infrastructure. With the rapid growth of digital data across domains such as healthcare, education, finance, and e-commerce, cloud platforms have become essential for scalable and cost-effective data management. However, as data outsourcing becomes more common, concerns regarding data security and privacy have intensified. Sensitive information stored in the cloud is often vulnerable to unauthorized access, data leakage, and misuse, especially when cloud service providers are not fully trusted. To address these challenges, encryption techniques are widely adopted, ensuring that data remains protected even when stored on external servers. While encryption guarantees confidentiality, it also introduces a major limitation: performing efficient search operations on encrypted data becomes difficult.

Searchable encryption has been proposed as a solution to this problem, allowing users to search encrypted data without decrypting it. Traditional searchable encryption techniques enable keyword-based search, where users generate search tokens that allow the cloud server to locate relevant documents without revealing the actual content. Although these methods improve usability, most of the existing solutions are designed for single data owner environments. In such systems, a single entity controls the data, encryption keys, and access permissions. This assumption does not hold true in modern cloud environments, where multiple independent users or organizations (tenants) store their data on shared infrastructure. In multi-tenant systems, each data owner operates independently, with different security policies and no willingness to share encryption keys. This creates a significant challenge when a user needs to perform a search across multiple data owners while preserving privacy and security.

2.2 LITERATURE SURVEY

The field of searchable encryption and secure cloud data retrieval has been widely explored over the past decade, particularly with the growing adoption of cloud computing and the increasing need for privacy-preserving mechanisms. One of the earliest and most influential works in this area was proposed by Dawn Xiaodong Song, David Wagner, and Adrian Perrig (2000), who introduced the concept of searchable encryption, allowing users to search over encrypted data without revealing its content. Their work laid the foundation for future research by demonstrating that it is possible to perform keyword searches securely, although the approach was limited in efficiency and scalability. Building upon this, Eu-Jin Goh (2003) proposed secure indexes using Bloom filters, which improved search efficiency but still faced challenges related to false positives and limited functionality.

Later, Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky (2006) introduced a more formal framework for Searchable Symmetric Encryption (SSE), providing strong security definitions and efficient constructions. Their work significantly improved the practicality of searchable encryption systems and became a benchmark for many subsequent studies. However, these early approaches primarily focused on single keyword search and single data owner environments. To address the need for more expressive queries, Cong Wang, Ning Cao, Jin Li, Kui Ren, and Wenjing Lou (2010) proposed a scheme for ranked keyword search over encrypted cloud data, which allowed users to retrieve documents in order of relevance. This work introduced the concept of ranking mechanisms using secure inner product computation, improving user experience by returning the most relevant results first.

As cloud computing evolved, researchers began to explore multi-keyword search techniques. Ning Cao, Cong Wang, Ming Li, Kui Ren, and Wenjing Lou (2011) proposed a privacy-preserving multi-keyword ranked search scheme, which enabled users to search using multiple keywords while maintaining data confidentiality. Their approach used

vector space models and secure k-nearest neighbor (kNN) techniques to compute similarity scores. Although effective, this method introduced additional computational overhead and did not address the challenges of multi-owner environments. Similarly, Jin Li, Qian Wang, Cong Wang, Ning Cao, Kui Ren, and Wenjing Lou (2011) worked on fuzzy keyword search, allowing for tolerance of minor typos and variations in search queries, further enhancing usability.

With the increasing complexity of cloud systems, the focus shifted toward supporting multiple data owners. Ming Li, Shucheng Yu, Ning Cao, and Wenjing Lou (2011) proposed a scheme for authorized private keyword search over encrypted data, which introduced fine-grained access control using attribute-based encryption. This allowed data owners to define access policies for different users. However, the system required complex key management and was not efficient for large-scale applications. In another significant contribution, Qian Wang, Cong Wang, Kui Ren, Ning Cao, and Wenjing Lou (2012) introduced mechanisms for secure and verifiable cloud storage, emphasizing the importance of ensuring data integrity and correctness. Their work highlighted the need for verification mechanisms, which were largely absent in earlier searchable encryption systems.

Further advancements were made by Peng Xu, Hai Jin, Qianhong Wu, and Wei Wang (2012), who explored public-key encryption with keyword search (PEKS) in multi-user settings. While their approach improved flexibility, it suffered from high computational costs and was not suitable for practical deployment in large-scale cloud environments. Around the same time, Seny Kamara and Kristin Lauter (2010) investigated cryptographic cloud storage, focusing on balancing efficiency and security. Their work emphasized the trade-offs between performance and privacy, which remains a critical challenge in this domain.

To address scalability and efficiency, David Cash, Stanislaw Jarecki, Charanjit Jutla, Hugo Krawczyk, Marcel-Cătălin Roșu, and Michael Steiner (2013) proposed dynamic searchable encryption schemes that support updates such as insertion and deletion of documents. This was an important step toward real-world applicability, as cloud data is

often dynamic. However, these systems still assumed a single data owner and did not consider cross-tenant search scenarios.

Similarly, Shi-Feng Sun, Joseph K. Liu, Amin Sakzad, Ron Steinfeld, and Dawu Gu (2017) introduced more efficient searchable encryption schemes with improved security guarantees, but the focus remained limited to single-owner models.

In the context of multi-owner environments, Bo Zhang, Fangguo Zhang, and Yu Chen (2016) proposed a multi-owner searchable encryption scheme, which allowed multiple data owners to upload encrypted data and support search operations. Although this approach addressed the multi-owner problem, it relied on shared system parameters and lacked mechanisms for verifying search results. Additionally, it did not provide accountability features, making it difficult to trace malicious users. Another notable contribution was made by Xiaofeng Chen, Jin Li, Jianfeng Ma, Qiang Tang, and Wenjing Lou (2014), who introduced verifiable searchable encryption, enabling users to verify the correctness of search results. This work was crucial in addressing the issue of untrusted cloud servers, but it did not incorporate user accountability or efficient parallel processing.

More recently, Ximeng Liu, Robert H. Deng, Kim-Kwang Raymond Choo, and Jian Weng (2015) explored privacy-preserving multi-keyword search with ranking, focusing on improving search accuracy and efficiency. Their approach combined secure indexing with ranking mechanisms, but it still faced challenges related to scalability and multi-tenant support. Similarly, Jianfeng Wang, Xiaofeng Chen, Jianfeng Ma, and Qiang Tang (2015) worked on secure multi-user searchable encryption schemes, emphasizing access control and user revocation. However, these systems often required complex key distribution mechanisms and did not fully address the independence of multiple data owners.

Despite these advancements, several research gaps remain. Most existing solutions are designed for single data owner scenarios and cannot be directly applied to multi-tenant cloud environments. Even in multi-owner schemes, issues such as key management, lack

of verification, absence of accountability, and inefficient search processing persist. Additionally, many systems assume that the cloud server is honest-but-curious, which is not always realistic in practical scenarios. The lack of mechanisms to verify search results and track user behavior poses significant security risks.

2.3 EXISTING SYSTEM

Existing systems suffer from several limitations when extended to multi-owner scenarios. First, they lack scalability, as users are required to send separate search queries to each data owner, increasing computational overhead and latency. Second, many solutions rely on a shared master key among all data owners, which is impractical and poses serious security risks. Third, these systems do not provide fine-grained access control, meaning users either gain full access to data or no access at all, without restrictions based on specific keywords. Furthermore, most existing approaches assume that the cloud server behaves honestly, even though it may be curious to learn sensitive information. As a result, there is no mechanism to verify whether the search results returned by the server are correct and complete. Additionally, user accountability is not addressed, making it impossible to trace misuse of search tokens or unauthorized sharing of access rights.

2.4 PROPOSED SYSTEM

The proposed system addresses these challenges by introducing a privacy-preserving ranked multi-keyword search framework designed specifically for multi-tenant cloud environments. The system is built upon a novel approach that combines searchable encryption with verification, accountability, and parallel processing capabilities.

The proposed system in your document builds upon these existing works and addresses their limitations by introducing a Verifiable and Accountable Keyword Searchable Encryption (VAKSE) scheme along with its parallel variant (PVAKSE).

Unlike earlier approaches, it supports multiple independent data owners without requiring shared keys, ensuring scalability and flexibility. It incorporates verification mechanisms

using message authentication codes to guarantee the correctness of search results, addressing the shortcomings of previous systems.

Furthermore, it introduces a novel accountability mechanism by embedding user identity into search tokens, enabling tracking of malicious activities. The addition of parallel processing enhances performance, making the system suitable for large-scale cloud environments.

2.5 FEASABILITY ANALYSIS

Compared to existing systems, the proposed solution offers several advantages. It eliminates the need for shared master keys, allowing data owners to maintain independence and control over their data. It enables efficient multi-keyword search across multiple data owners with a single query, reducing computational overhead. The inclusion of verification and accountability mechanisms ensures trustworthiness and security, addressing the limitations of traditional approaches. Furthermore, the use of parallel processing enhances scalability and performance, making the system practical for large-scale cloud environments.

The increasing reliance on cloud computing for data storage and management has created a need for secure and efficient search mechanisms that preserve data privacy. Existing searchable encryption techniques are not sufficient to handle the complexities of multi-tenant environments, where multiple independent data owners coexist. The proposed system provides a comprehensive solution by integrating privacy preservation, verifiability, accountability, and performance optimization into a unified framework. Through real-world case studies in healthcare and finance, it is evident that such a system is essential for enabling secure data sharing and analysis in modern cloud-based applications.

This approach not only enhances data security but also improves usability and efficiency, paving the way for more reliable and trustworthy cloud computing systems.

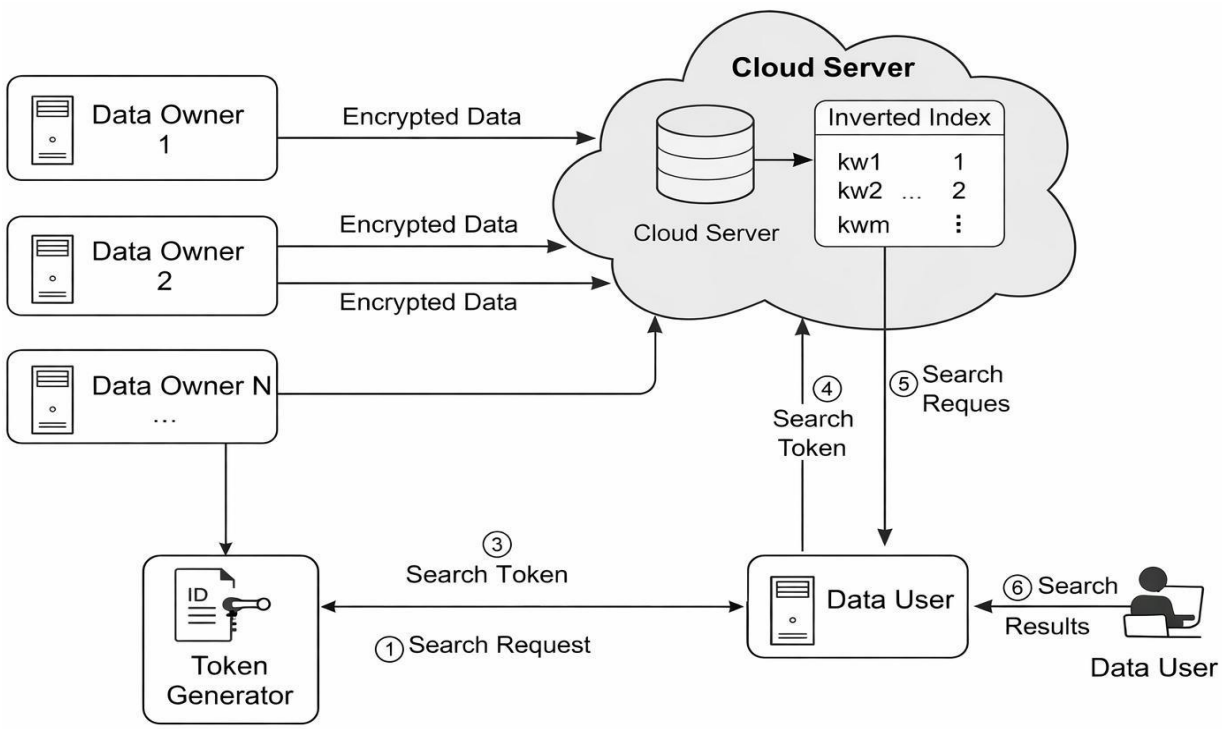
CHAPTER-3

DESIGN

3.1 SYSTEM DESIGN

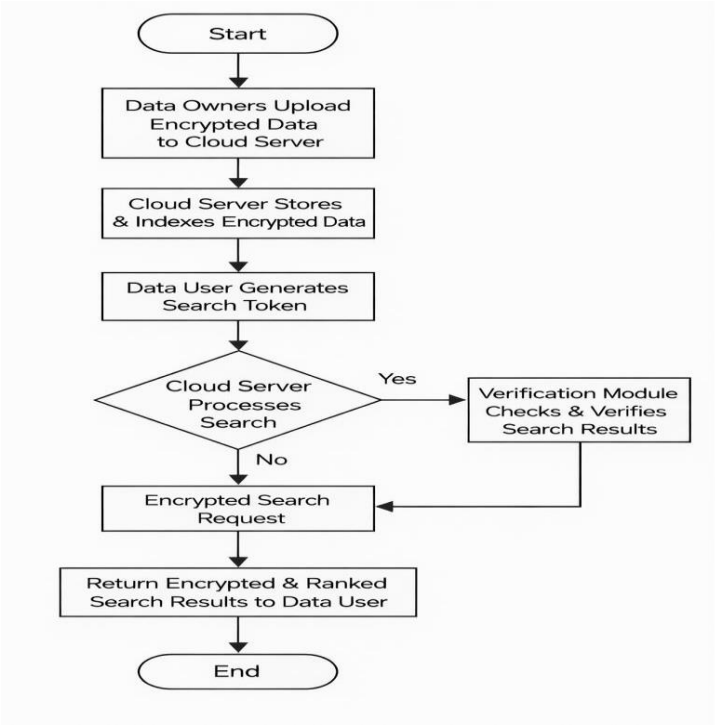
The system follows a secure cloud-based architecture that enables privacy-preserving keyword search over encrypted data. The architecture includes multiple independent data owners, cloud servers, and users who perform secure search operations without revealing sensitive data.

ARCHITECHTURE

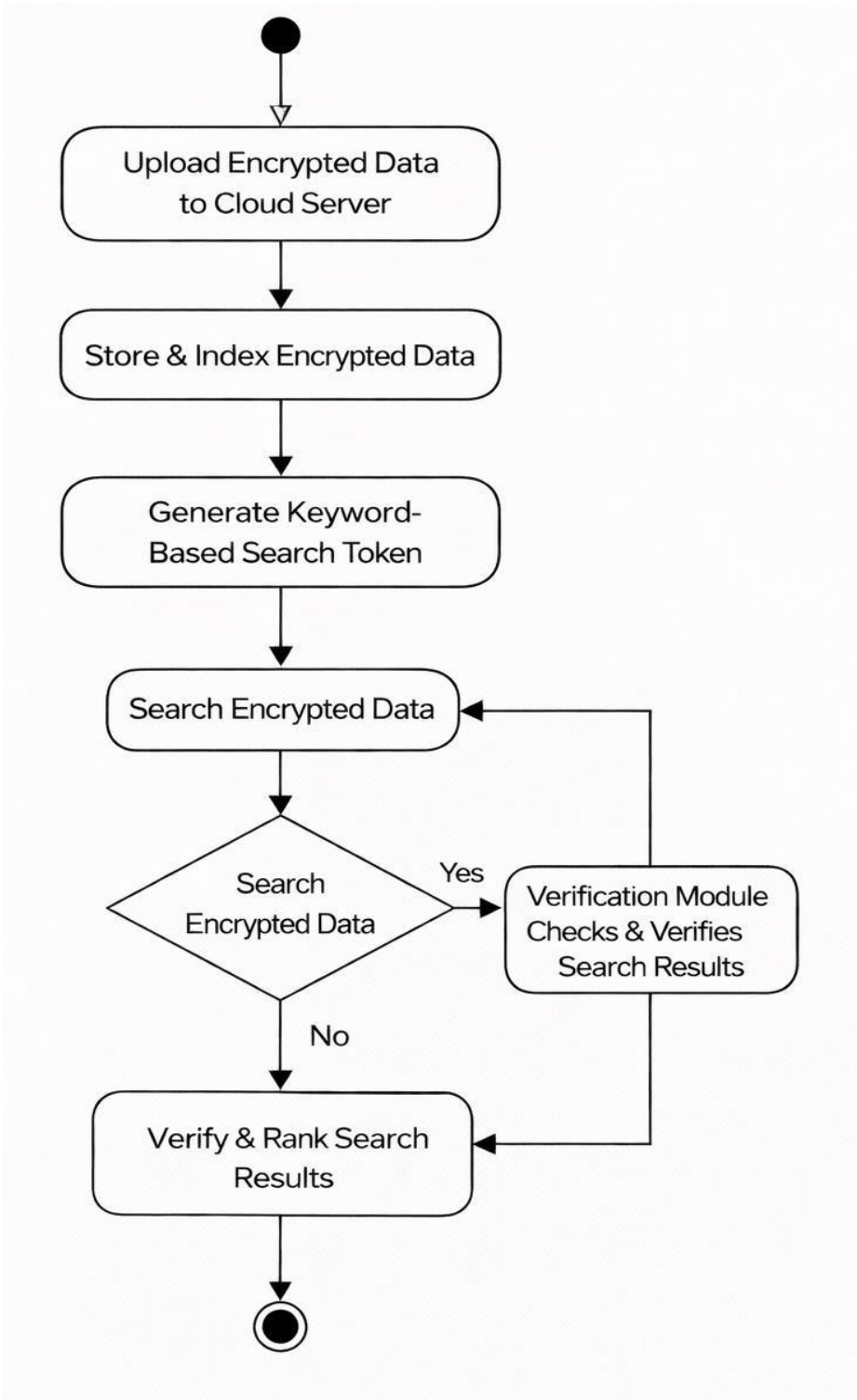


DATA FLOW

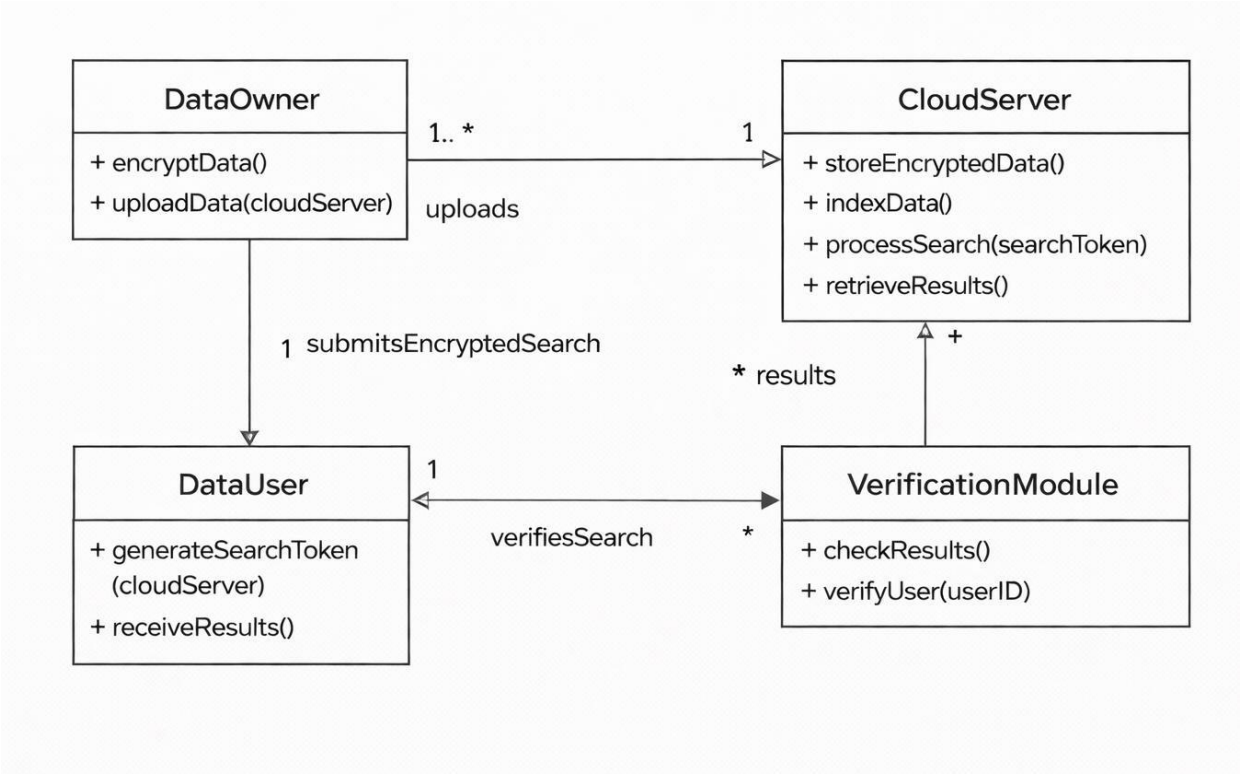
FLOW CHART



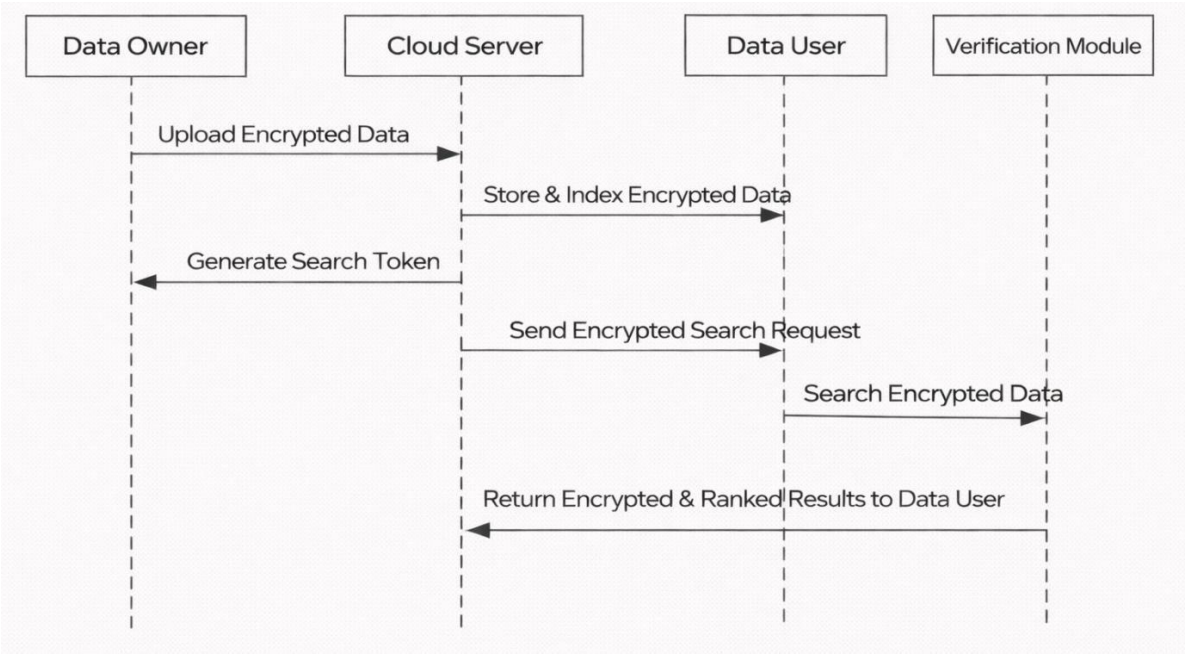
ACTIVITY DIAGRAM



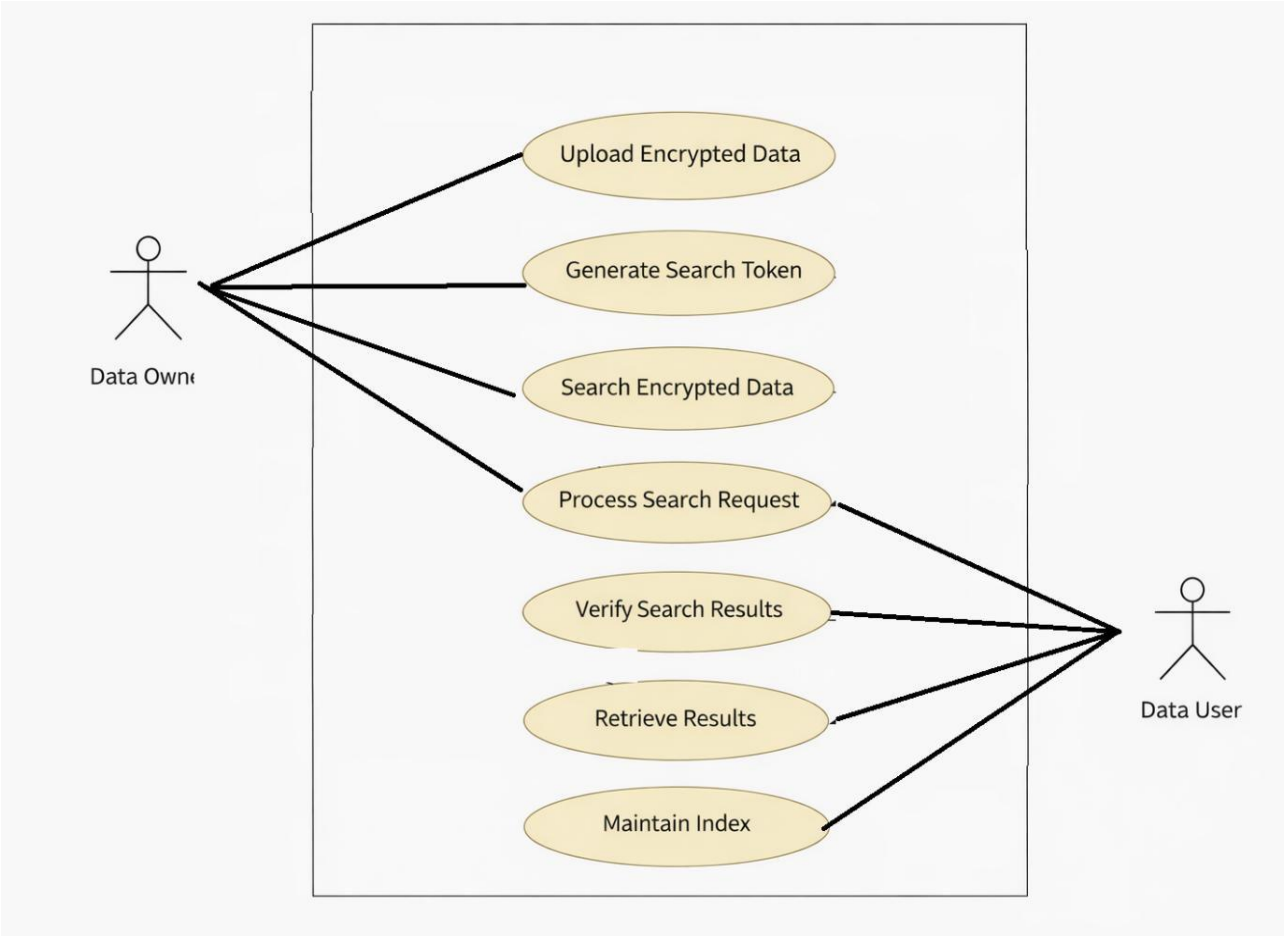
CLASS DIAGRAM



SEQUENCE DIAGRAM



USE CASE


Upload Encrypted Data
Generate Search Token
Search Encrypted Data
Process Search Request
Verify Search Results
Retrieve Results
Maintain Index
Data Owne
Data User

3.1.1 Description about Modules

1. **Data Owner Module**

   The data owner uploads documents to the cloud server after encrypting them to maintain privacy. It also generates a searchable index for keywords so that encrypted files can be searched securely.

2. **Data Encryption Module**

   This module encrypts the documents before storing them in the cloud environment. Encryption ensures that sensitive information remains protected from unauthorized access.

3. **Cloud Server Module**

   The cloud server stores encrypted files and maintains the keyword-based inverted index. It processes search requests from users and retrieves relevant encrypted documents.

4. **Token Generation Module**

   In this module, a secure search token is generated based on the user's keyword request. The token also embeds the user identity to maintain accountability and prevent misuse.

5. **Keyword Search Module**

   This module performs multi-keyword search on encrypted data stored in the cloud. It matches the generated search token with the encrypted index to find relevant documents.

6. **Verification Module**

   The verification module checks the correctness and completeness of search results. It uses Message Authentication Codes (MAC) to ensure the cloud server returns valid results.

7. **Result Retrieval Module**

   This module sends the ranked encrypted search results back to the user. The user decrypts the results and views the required documents securely.

3.2 Detailed Design of the Project

The system is designed using a three-tier architecture consisting of:

- **User Interface Layer**: Built using HTML, JavaScript, and JSP for interaction.
- **Application Layer**: Developed using Java Servlets and business logic modules.
- **Database Layer**: Uses MySQL with JDBC connectivity.

The system ensures:

- Secure encryption and storage
- Efficient keyword-based search
- Verification and accountability
- Parallel processing for performance

CHAPTER-4

IMPLEMENTATION

4.1 REQUIREMENTS

4.1.1 H/W REQUIREMENTS

Processor	:	Pentium / Intel Core
RAM	:	4 GB (Min)
Hard Disk	:	20 GB

4.1.2 S/W REQUIREMENTS

Operating System (OS)	:	Windows Family
Coding Language	:	Java
Database	:	My SQL

➡ SOFTWARE DESCRIPTION

Java Technology

Java technology is both a programming language and a platform.

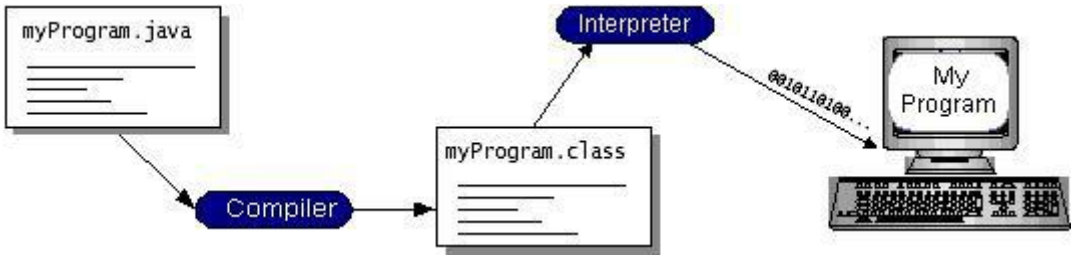
The Java Programming Language

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented

- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

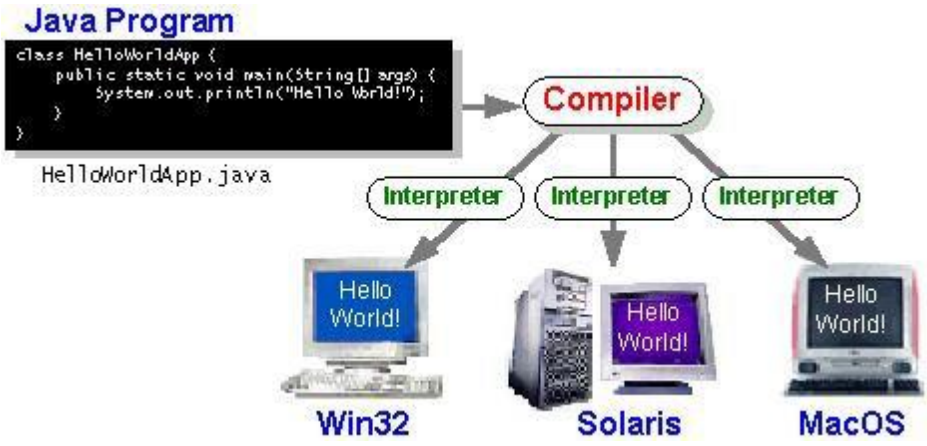
With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called Java byte codes —the platform-independent codes interpreted by the interpreter on the Java platform. The interpreter parses and runs each Java byte code instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed. The following figure illustrates how this works.



You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java byte codes help make "write once, run anywhere" possible. You can compile your program into byte

codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM.

That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.



The Java Platform

A platform is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware-based platforms.

The Java platform has two components:

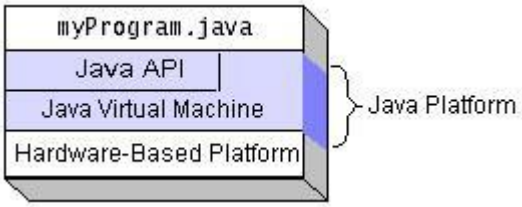
- The Java Virtual Machine (Java VM)
- The Java Application Programming Interface (Java API)

You've already been introduced to the Java VM. It's the base for the Java platform and is ported onto various hardware-based platforms.

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and interfaces; these libraries are known as packages. The next section, What Can Java Technology Do? Highlights what

functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.



Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart compilers, well-tuned interpreters, and just-in-time byte code compilers can bring performance close to that of native code without threatening portability.

What Can Java Technology Do?

The most common types of programs written in the Java programming language are applets and applications. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser.

However, the Java programming language is not just for writing cute, entertaining applets for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

An application is a standalone program that runs directly on the Java platform. A special kind of application known as a server serves and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a servlet. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets

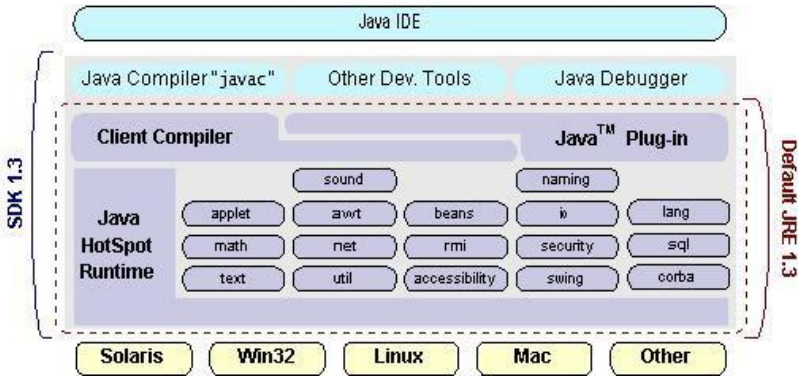
run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

The essentials: Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.

- **Applets**: The set of conventions used by applets.
- **Networking**: URLs, TCP (Transmission Control Protocol), UDP (User Data gram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization**: Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.
- **Security**: Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components**: Known as JavaBeans™, can plug into existing component architectures.
- **Object serialization**: Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- **Java Database Connectivity (JDBC™)**: Provides uniform access to a wide range of relational databases.

The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more. The following figure depicts what is included in the Java 2 SDK.



How Will Java Technology Change My Life?

We can't promise you fame, fortune, or even a job if you learn the Java programming language. Still, it is likely to make your programs better and requires less effort than other languages. We believe that Java technology will help you do the following:

- **Get started quickly**: Although the Java programming language is a powerful object-oriented language, it's easy to learn, especially for programmers already familiar with C or C++.

- **Write less code**: Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.

- **Write better code**: The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory leaks. Its object orientation, its JavaBeans component architecture, and its wide-ranging, easily extendible API let you reuse other people's tested code and introduce fewer bugs.

- **Develop programs more quickly**: Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.

- **Avoid platform dependencies with 100% Pure Java**: You can keep your program portable by avoiding the use of libraries written in other languages. The 100%

Pure Java™ Product Certification Program has a repository of historical process manuals, white papers, brochures, and similar materials online.

- **Write once, run anywhere**: Because 100% Pure Java programs are compiled into machine-independent byte codes, they run consistently on any Java platform.

- **Distribute software more easily**: You can upgrade applets easily from a central server. Applets take advantage of the feature of allowing new classes to be loaded "on the fly," without recompiling the entire program.

ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a de facto standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE.

There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC datasources.

From a programming perspective, the beauty of ODBC is that the application can be written to use the same set of function calls to interface with any data source, regardless of the database vendor. The source code of the application doesn't change whether it talks to Oracle or SQL Server. We only mention these two as an example. There are ODBC drivers available for several dozen popular database systems. Even Excel spreadsheets and plain text files can be turned into data sources. The operating system uses the Registry information written by ODBC Administrator to determine which low-level ODBC drivers are needed to talk to the data source (such as the interface to Oracle or SQL Server). The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server environment, the ODBC API even handles many of the network issues for the application programmer.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface.

ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year.

JDBC

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of "plug-in" database connectivity modules, or drivers. If a database vendor wishes to have JDBC support, he or she must provide the driver for each platform that the database and Java run on.

To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you

discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution.

JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

JDBC Goals

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java.

The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do. The eight design goals for JDBC are as follows:

1. SQL Level API

The designers felt that their main goal was to define a SQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently. Attaining this goal allows for future tool vendors to "generate" JDBC code and to hide many of JDBC's complexities from the end user.

2. SQL Conformance

SQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed

through it to the underlying database driver. This allows the connectivity module to handle non-standard functionality in a manner that is suitable for its users.

1. **Provide a Java interface that is consistent with the rest of the Java system**

Because of Java's acceptance in the user community thus far, the designers feel that they should not stray from the current design of the core Java system.

1. **Keep it simple**

This goal probably appears in all software design goal listings. JDBC is no exception. Sun felt that the design of JDBC should be very simple, allowing for only one method of completing a task per mechanism. Allowing duplicate functionality only serves to confuse the users of the API.

Use strong, static typing wherever possible

Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

Keep the common cases simple

Because more often than not, the usual SQL calls used by the programmer are simple SELECT's, INSERT's, DELETE's and UPDATE's, these queries should be simple to perform with JDBC. However, more complex SQL statements should also be possible. Finally we decided to proceed the implementation using Java Networking.

And for dynamically updating the cache table we go for MS Access database.

Java ha two things: a programming language and a platform.

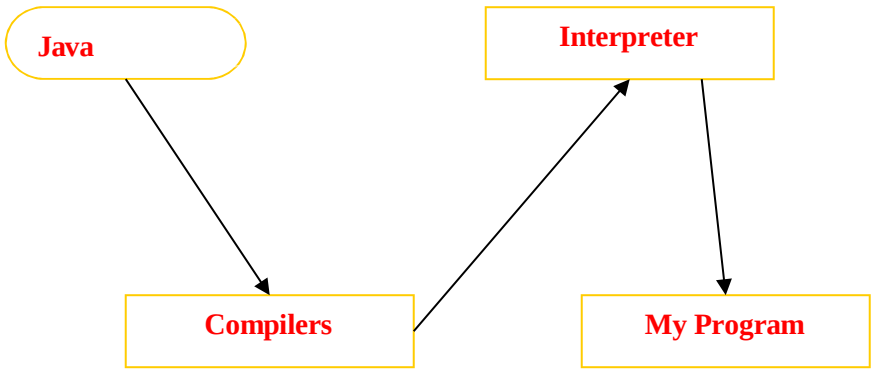
Java is a high-level programming language that is all of the following

Simple	Architecture-neutral
Object-oriented	Portable

Distributed	High-performance
Interpreted	Multithreaded
Robust	Dynamic
Secure	

Java is also unusual in that each Java program is both compiled and interpreted. With a compile you translate a Java program into an intermediate language called Java byte codes the platform-independent code instruction is passed and run on the computer.

Compilation happens just once; interpretation occurs each time the program is executed. The figure illustrates how this works.



You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a Java development tool or a Web browser that can run Java applets, is an implementation of the Java VM. The Java VM can also be implemented in hardware.

Java byte codes help make "write once, run anywhere" possible. You can compile your Java program into byte codes on my platform that has a Java compiler. The byte codes can then be run any implementation of the Java VM. For example, the same Java

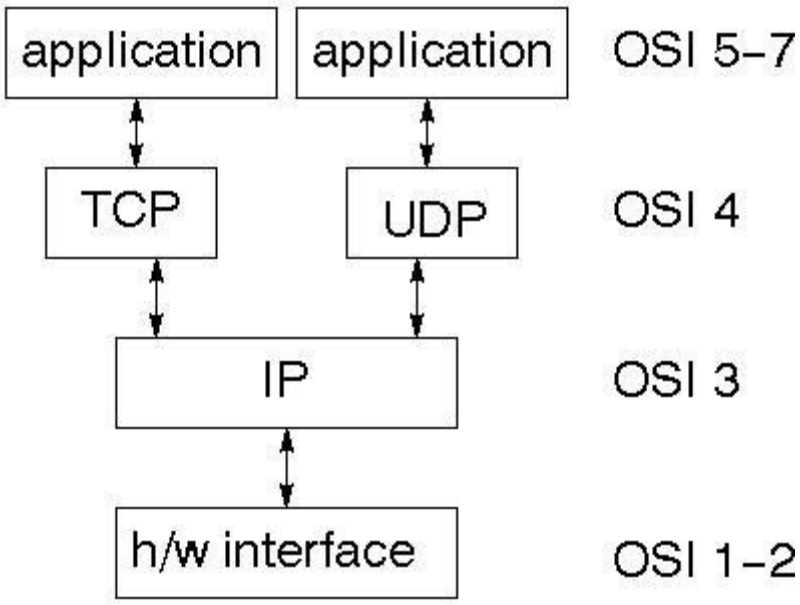
program

can run Windows NT, Solaris, and Macintosh.

NETWORKING

TCP/IP stack

The TCP/IP stack is shorter than the OSI one:



TCP is a connection-oriented protocol; UDP (User Datagram Protocol) is a connectionless protocol.

IP datagram's

The IP layer provides a connectionless and unreliable delivery system. It considers each datagram independently of the others. Any association between datagram must be supplied by the higher layers. The IP layer supplies a checksum that includes its own header. The header includes the source and destination addresses. The IP layer handles routing through an Internet. It is also responsible for breaking up large datagram into

smaller ones for transmission and reassembling them at the other end.

UDP

UDP is also connectionless and unreliable. What it adds to IP is a checksum for the contents of the datagram and port numbers. These are used to give a client/server model - see later.

TCP

TCP supplies logic to give a reliable connection-oriented protocol above IP. It provides a virtual circuit that two processes can use to communicate.

Internet addresses

In order to use a service, you must be able to find it. The Internet uses an address scheme for machines so that they can be located. The address is a 32 bit integer which gives the IP address. This encodes a network ID and more addressing. The network ID falls into various classes according to the size of the network address.

Network address

Class A uses 8 bits for the network address with 24 bits left over for other addressing. Class B uses 16 bit network addressing. Class C uses 24 bit network addressing and class D uses all 32.

Subnet address

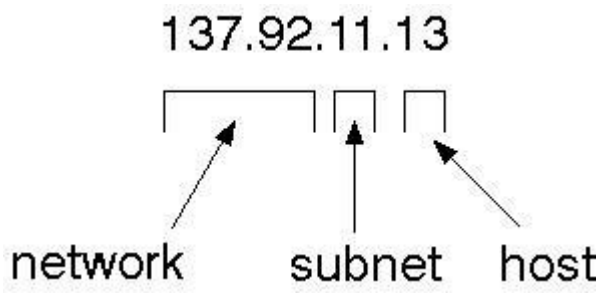
Internally, the UNIX network is divided into sub networks. Building 11 is currently on one sub network and uses 10-bit addressing, allowing 1024 different hosts.

Host address

8 bits are finally used for host addresses within our subnet. This places a limit of 256

machines that can be on the subnet.

Total address



The 32 bit address is usually written as 4 integers separated by dots.

Port addresses

A service exists on a host, and is identified by its port. This is a 16 bit number. To send a message to a server, you send it to the port for that service of the host that it is running on. This is not location transparency! Certain of these ports are "well known".

Sockets

A socket is a data structure maintained by the system to handle network connections. A socket is created using the call socket. It returns an integer that is like a file descriptor. In fact, under Windows, this handle can be used with Read File and Write File functions.

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int family, int type, int protocol);
```

Here "family" will be AF_INET for IP communications, protocol will be zero, and type will depend on whether TCP or UDP is used. Two processes wishing to communicate over a network create a socket each. These are similar to two ends of a pipe - but the actual pipe does not yet exist.

JFree Chart

JFreeChart is a free 100% Java chart library that makes it easy for developers to display professional quality charts in their applications. JFreeChart's extensive feature set includes:
A consistent and well-documented API, supporting a wide range of chart types;
A flexible design that is easy to extend, and targets both server-side and client-side applications;
Support for many output types, including Swing components, image files (including PNG and JPEG), and vector graphics file formats (including PDF, EPS and SVG);
JFreeChart is "open source" or, more specifically, free software. It is distributed under the terms of the GNU Lesser General Public Licence (LGPL), which permits use in proprietary applications.

1. Map Visualizations

Charts showing values that relate to geographical areas. Some examples include: (a) population density in each state of the United States, (b) income per capita for each country in Europe, (c) life expectancy in each country of the world. The tasks in this project include:

Sourcing freely redistributable vector outlines for the countries of the world, states/provinces in particular countries (USA in particular, but also other areas);
Creating an appropriate dataset interface (plus default implementation), a rendered, and integrating this with the existing XYPlot class in JFreeChart;
Testing, documenting, testing some more, documenting some more.

2. Time Series Chart Interactivity

Implement a new (to JFreeChart) feature for interactive time series charts --- to display a separate control that shows a small version of ALL the time series data, with a sliding

"view" rectangle that allows you to select the subset of the time series data to display in the main chart.

3. Dashboards

There is currently a lot of interest in dashboard displays. Create a flexible dashboard mechanism that supports a subset of JFreeChart chart types (dials, pies, thermometers, bars, and lines/time series) that can be delivered easily via both Java Web Start and an applet.

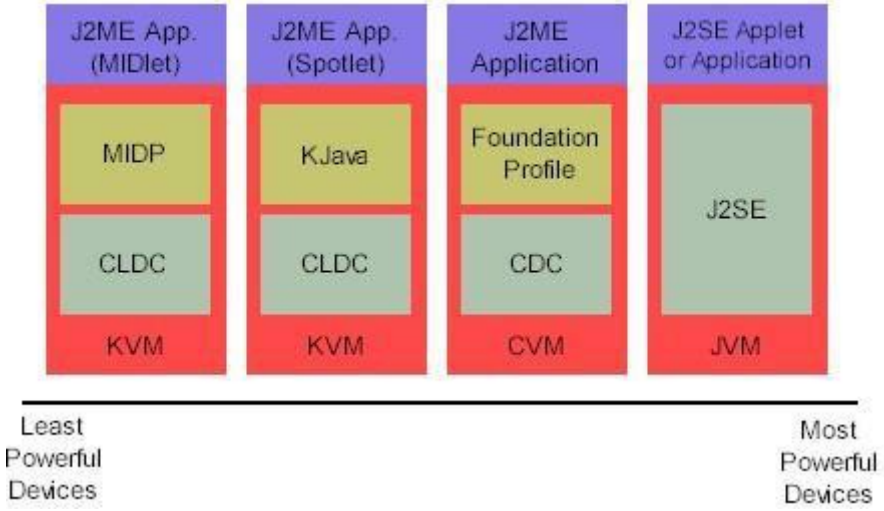
4. Property Editors

The property editor mechanism in JFreeChart only handles a small subset of the properties that can be set for charts. Extend (or reimplement) this mechanism to provide greater end-user control over the appearance of the charts.

J2ME (Java 2 Micro edition):-

Sun Microsystems defines J2ME as "a highly optimized Java run-time environment targeting a wide range of consumer products, including pagers, cellular phones, screen-phones, digital set-top boxes and car navigation systems." Announced in June 1999 at the JavaOne Developer Conference, J2ME brings the cross-platform functionality of the Java language to smaller devices, allowing mobile wireless devices to share applications. With J2ME, Sun has adapted the Java platform for consumer products that incorporate or are based on small computing devices.

1.General J2ME architecture



J2ME uses configurations and profiles to customize the Java Runtime Environment (JRE). As a complete JRE, J2ME is comprised of a configuration, which determines the JVM used, and a profile, which defines the application by adding domain-specific classes. The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. We'll discuss configurations in detail in the The profile defines the application; specifically, it adds domain-specific classes to the J2ME configuration to define certain uses for devices. We'll cover profiles in depth in the The following graphic depicts the relationship between the different virtual machines, configurations, and profiles. It also draws a parallel with the J2SE API and its Java virtual machine. While the J2SE virtual machine is generally referred to as a JVM, the J2ME virtual machines, KVM and CVM, are subsets of JVM. Both KVM and CVM can be thought of as a kind of Java virtual machine -- it's just that they are shrunken versions of the J2SE JVM and are specific to J2ME.

2.Developing J2ME applications

Introduction In this section, we will go over some considerations you need to keep in mind when developing applications for smaller devices. We'll take a look at the way the compiler is invoked when using J2SE to compile J2ME applications. Finally, we'll explore packaging and deployment and the role preverification plays in this process.

3.Design considerations for small devices

Developing applications for small devices requires you to keep certain strategies in mind during the design phase. It is best to strategically design an application for a small device before you begin coding. Correcting the code because you failed to consider all of the "gotchas" before developing the application can be a painful process. Here are some design strategies to consider:

* Keep it simple. Remove unnecessary features, possibly making those features a separate, secondary application.

* Smaller is better. This consideration should be a "no brainer" for all developers. Smaller applications use less memory on the device and require shorter installation times. Consider packaging your Java applications as compressed Java Archive (jar) files.

* Minimize run-time memory use. To minimize the amount of memory used at run time, use scalar types in place of object types. Also, do not depend on the garbage collector. You should manage the memory efficiently yourself by setting object references to null when you are finished with them. Another way to reduce run-time memory is to use lazy instantiation, only allocating objects on an as-needed basis. Other ways of reducing overall and peak memory use on small devices are to release resources quickly, reuse objects, and avoid exceptions.

4.Configurations overview

The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. Currently, two configurations exist for J2ME, though others may be defined in the future:

* **Connected Limited Device Configuration (CLDC)** is used specifically with the

KVM for 16-bit or 32-bit devices with limited amounts of memory. This is the configuration (and the virtual machine) used for developing small J2ME applications. Its size limitations make CLDC more interesting and challenging (from a development point of view) than CDC. CLDC is also the configuration that we will use for developing our drawing tool application. An example of a small wireless device running small applications is a Palm hand-held computer.

* **Connected Device Configuration (CDC)** is used with the C virtual machine (CVM) and is used for 32-bit architectures requiring more than 2 MB of memory. An example of such a device is a Net TV box.

5.J2ME profiles

What is a J2ME profile?

As we mentioned earlier in this tutorial, a profile defines the type of device supported. The Mobile Information Device Profile (MIDP), for example, defines classes for cellular phones. It adds domain-specific classes to the J2ME configuration to define uses for similar devices. Two profiles have been defined for J2ME and are built upon CLDC: KJava and MIDP. Both KJava and MIDP are associated with CLDC and smaller devices. Profiles are built on top of configurations. Because profiles are specific to the size of the device (amount of memory) on which an application runs, certain profiles are associated with certain configurations.

A skeleton profile upon which you can create your own profile, the Foundation Profile, is available for CDC.

Profile 1: KJava

KJava is Sun's proprietary profile and contains the KJava API. The KJava profile is built on top of the CLDC configuration. The KJava virtual machine, KVM, accepts the same byte codes and class file format as the classic J2SE virtual machine. KJava contains a Sun-specific API that runs on the Palm OS. The KJava API has a great deal in common with the J2SE Abstract Windowing Toolkit (AWT). However, because it is not a standard J2ME package, its main package is com.sun.kjava. We'll learn more about the KJava API

later in this tutorial when we develop some sample applications.

Profile 2: MIDP

MIDP is geared toward mobile devices such as cellular phones and pagers. The MIDP, like KJava, is built upon CLDC and provides a standard run-time environment that allows new applications and services to be deployed dynamically on end user devices. MIDP is a common, industry-standard profile for mobile devices that is not dependent on a specific vendor. It is a complete and supported foundation for mobile application development. MIDP contains the following packages, the first three of which are core CLDC packages, plus three MIDP-specific packages.

* java.lang
* java.io
* java.util
* javax.microedition.io
* javax.microedition.lcdui
* javax.microedition.midlet
* javax.microedition.rms

CLIENT SERVER

Over view:

With the varied topic in existence in the fields of computers, Client Server is one, which has generated more heat than light, and also more hype than reality. This technology has acquired a certain critical mass attention with its dedication conferences and magazines. Major computer vendors such as IBM and DEC, have declared that Client Servers is their main future market. A survey of DBMS magazine reveled that 76% of its readers were actively looking at the client server solution. The growth in the client server development tools from $200 million in 1992 to more than $1.2 billion in 1996.

Client server implementations are complex but the underlying concept is simple and powerful. A client is an application running with local resources but able to request the

database and relate the services from separate remote server. The software mediating this client server interaction is often referred to as MIDDLEWARE.

The typical client either a PC or a Work Station connected through a network to a more powerful PC, Workstation, Midrange or Main Frames server usually capable of handling request from more than one client. However, with some configuration server may also act as client. A server may need to access other server in order to process the original client request.

The key client server idea is that client as user is essentially insulated from the physical location and formats of the data needs for their application. With the proper middleware, a client input from or report can transparently access and manipulate both local database on the client machine and remote databases on one or more servers. An added bonus is the client server opens the door to multi-vendor database access indulging heterogeneous table joins.

What is a Client Server

Two prominent systems in existence are client server and file server systems. It is essential to distinguish between client servers and file server systems. Both provide shared network access to data but the comparison dens there! The file server simply provides a remote disk drive that can be accessed by LAN applications on a file by file basis. The client server offers full relational database services such as SQL-Access, Record modifying, Insert, Delete with full relational integrity backup/ restore performance for high volume of transactions, etc. the client server middleware provides a flexible interface between client and server, who does what, when and to whom.

Why Client Server

Client server has evolved to solve a problem that has been around since the earliest days of computing: how best to distribute your computing, data generation and data storage resources in order to obtain efficient, cost effective departmental an enterprise wide data processing. During mainframe era choices were quite limited. A central machine housed both the CPU and DATA (cards, tapes, drums and later disks). Access to these resources

was initially confined to batched runs that produced departmental reports at the appropriate intervals. A strong central information service department ruled the corporation. The role of the rest of the corporation limited to requesting new or more frequent reports and to provide hand written forms from which the central data banks were created and updated. The earliest client server solutions therefore could best be characterized as “SLAVE-MASTER”.

Time-sharing changed the picture. Remote terminal could view and even change the central data, subject to access permissions. And, as the central data banks evolved in to sophisticated relational database with non-programmer query languages, online users could formulate adhoc queries and produce local reports with out adding to the MIS applications software backlog. However remote access was through dumb terminals, and the client server remained subordinate to the Slave\Master.

FRONT END Or UESR INTERFACE DESIGN

The entire user interface is planned to be developed in browser specific environment with a touch of Intranet-Based Architecture for achieving the Distributed Concept.

The browser specific components are designed by using the HTML standards, and the dynamism of the designed by concentrating on the constructs of the Java Server Pages.

COMMUNICATION Or DATA BASE TIER

The Communication architecture is designed by concentrating on the Standards of Servlets and Enterprise Java Beans. The database connectivity is established by using the Java Data Base Connectivity.

The standards of three-tire architecture are given major concentration to keep the standards of higher cohesion and limited coupling for effectiveness of the operations.

FEATURES OF LANGUAGE USED

In my project, I have chosen Java language for developing the code.

About Java

Initially the language was called as “oak” but it was renamed as “Java” in 1995. The primary motivation of this language was the need for a platform-independent (i.e., architecture neutral) language that could be used to create software to be embedded in various consumer electronic devices.

- Java is a programmer’s language.
- Java is cohesive and consistent.
- Except for those constraints imposed by the Internet environment, Java gives the programmer, full control.

Finally, Java is to Internet programming where C was to system programming.

Importance of Java to the Internet

Java has had a profound effect on the Internet. This is because; Java expands the Universe of objects that can move about freely in Cyberspace. In a network, two categories of objects are transmitted between the Server and the Personal computer. They are: Passive information and Dynamic active programs. The Dynamic, Self-executing programs cause serious problems in the areas of Security and probability. But, Java addresses those concerns and by doing so, has opened the door to an exciting new form of program called the Applet.

Java can be used to create two types of programs

Applications and Applets: An application is a program that runs on our Computer under the operating system of that computer. It is more or less like one creating using C or C++. Java’s ability to create Applets makes it important. An Applet is an application designed to be transmitted over the Internet and executed by a Java –compatible web browser. An applet is actually a tiny Java program, dynamically downloaded across the network, just like an image. But the difference is, it is an intelligent program, not just a media file. It can react to the user input and dynamically change.

FEATURES OF JAVA

Security

Every time you that you download a “normal” program, you are risking a viral infection. Prior to Java, most users did not download executable programs frequently, and those who did scanned them for viruses prior to execution. Most users still worried about the possibility of infecting their systems with a virus. In addition, another type of malicious program exists that must be guarded against. This type of program can gather private information, such as credit card numbers, bank account balances, and passwords. Java answers both these concerns by providing a “firewall” between a network application and your computer.

When you use a Java-compatible Web browser, you can safely download Java applets without fear of virus infection or malicious intent.

Portability

For programs to be dynamically downloaded to all the various types of platforms connected to the Internet, some means of generating portable executable code is needed .As you will see, the same mechanism that helps ensure security also helps create portability. Indeed, Java’s solution to these two problems is both elegant and efficient.

The Byte code

The key that allows the Java to solve the security and portability problems is that the output of Java compiler is Byte code. Byte code is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM). That is, in its standard form, the JVM is an interpreter for byte code.

Translating a Java program into byte code helps makes it much easier to run a program in a wide variety of environments. The reason is, once the run-time package exists for a given system, any Java program can run on it.

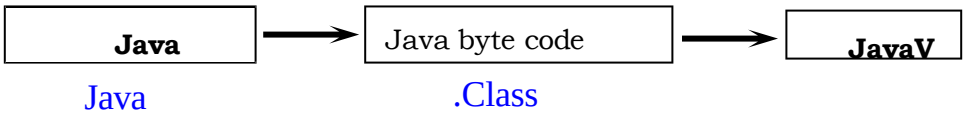
Although Java was designed for interpretation, there is technically nothing about Java

that prevents on-the-fly compilation of byte code into native code. Sun has just completed its Just In Time (JIT) compiler for byte code. When the JIT compiler is a part of JVM, it compiles byte code into executable code in real time, on a piece-by-piece, demand basis. It is not possible to compile an entire Java program into executable code all at once, because Java performs various run-time checks that can be done only at run time. The JIT compiles code, as it is needed, during execution.

JAVA VIRTUAL MACHINE (JVM)

Beyond the language, there is the Java virtual machine. The Java virtual machine is an important element of the Java technology. The virtual machine can be embedded within a web browser or an operating system. Once a piece of Java code is loaded onto a machine, it is verified. As part of the loading process, a class loader is invoked and does byte code verification makes sure that the code that's has been generated by the compiler will not corrupt the machine that it's loaded on. Byte code verification takes place at the end of the compilation process to make sure that is all accurate and correct. So byte code verification is integral to the compiling and executing of Java code.

Overall Description



Picture showing the development process of JAVA Program

Java programming uses to produce byte codes and executes them. The first box indicates that the Java source code is located in a. Java file that is processed with a Java compiler called javac. The Java compiler produces a file called a. class file, which contains the byte code. The. Class file is then loaded across the network or loaded locally on your machine into the execution environment is the Java virtual machine, which interprets and executes the byte code.

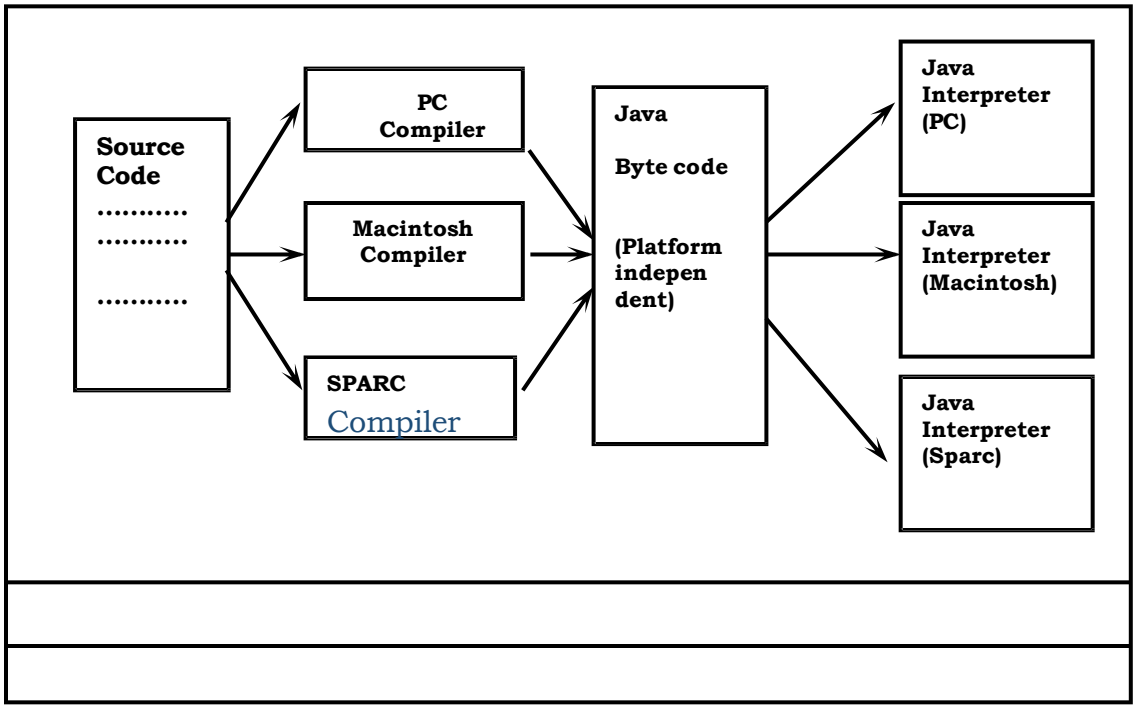
Java Architecture

Java architecture provides a portable, robust, high performing environment for development. Java provides portability by compiling the byte codes for the Java Virtual Machine, which is then interpreted on each platform by the run-time environment. Java is a dynamic system, able to load code when needed from a machine in the same room or across the planet.

Compilation of code

When you compile the code, the Java compiler creates machine code (called byte code) for a hypothetical machine called Java Virtual Machine (JVM). The JVM is supposed to execute the byte code. The JVM is created for overcoming the issue of portability. The code is written and compiled for one machine and interpreted on all machines. This machine is called Java Virtual Machine.

Compiling and interpreting Java Source Code



During run-time the Java interpreter tricks the byte code file into thinking that it is

running on a Java Virtual Machine. In reality this could be a Intel Pentium Windows 95 or Sun SARC station running Solaris or Apple Macintosh running system and all could receive code from any computer through Internet and run the Applets.

Simple

Java was designed to be easy for the Professional programmer to learn and to use effectively. If you are an experienced C++ programmer, learning Java will be even easier. Because Java inherits the C/C++ syntax and many of the object oriented features of C++. Most of the confusing concepts from C++ are either left out of Java or implemented in a cleaner, more approachable manner. In Java there are a small number of clearly defined ways to accomplish a given task.

Object-Oriented

Java was not designed to be source-code compatible with any other language. This allowed the Java team the freedom to design with a blank slate. One outcome of this was a clean usable, pragmatic approach to objects. The object model in Java is simple and easy to extend, while simple types, such as integers, are kept as high-performance non-objects.

Robust

The multi-platform environment of the Web places extraordinary demands on a program, because the program must execute reliably in a variety of systems. The ability to create robust programs was given a high priority in the design of Java. Java is strictly typed language; it checks your code at compile time and run time.

Java virtually eliminates the problems of memory management and deallocation, which is completely automatic. In a well-written Java program, all run time errors can –and should –be managed by your program.

JAVASCRIPT

JavaScript is a script-based programming language that was developed by Netscape

Communication Corporation. JavaScript was originally called Live Script and renamed as JavaScript to indicate its relationship with Java. JavaScript supports the development of both client and server components of Web-based applications. On the client side, it can be used to write programs that are executed by a Web browser within the context of a Web page. On the server side, it can be used to write Web server programs that can process information submitted by a Web browser and then updates the browser's display accordingly

Even though JavaScript supports both client and server Web programming, we prefer JavaScript at Client side programming since most of the browsers supports it. JavaScript is almost as easy to learn as HTML, and JavaScript statements can be included in HTML documents by enclosing the statements between a pair of scripting tags

```
<SCRIPTS>..</SCRIPT>.

<SCRIPT LANGUAGE = "JavaScript">

JavaScript statements

</SCRIPT>
```

Here are a few things we can do with JavaScript :

- Validate the contents of a form and make calculations.
- Add scrolling or changing messages to the Browser's status line.
- Animate images or rotate images that change when we move the mouse over them.
- Detect the browser in use and display different content for different browsers.

- Detect installed plug-ins and notify the user if a plug-in is required.

We can do much more with JavaScript, including creating entire application.

J A V A S C R I P T V s J A V A

JavaScript and Java are entirely different languages. A few of the most glaring differences are:

- Java applets are generally displayed in a box within the web document; JavaScript can affect any part of the Web document itself.
- While JavaScript is best suited to simple applications and adding interactive features to Web pages; Java can be used for incredibly complex applications.

There are many other differences but the important thing to remember is that JavaScript and Java are separate languages. They are both useful for different things; in fact they can be used together to combine their advantages.

There are many other differences but the important thing to remember is that JavaScript and Java are separate languages. They are both useful for different things; in fact they can be used together to combine their advantages.

A D V A N T A G E S

- JavaScript can be used for Sever-side and Client-side scripting.
- It is more flexible than VBScript.
- JavaScript is the default scripting languages at Client-side since all the browsers supports it.

Hyper Text Markup Language

Hypertext Markup Language **(HTML)**, the languages of the World Wide Web (WWW), allows users to produces Web pages that include text, graphics and pointer to other Web pages (Hyperlinks).

HTML is not a programming language but it is an application of ISO Standard 8879, SGML (Standard Generalized Markup Language), but specialized to hypertext and adapted to the Web. The idea behind Hypertext is that instead of reading text in rigid linear structure, we can easily jump from one point to another point. We can navigate through the information based on our interest and preference. A markup language is simply a series of elements, each delimited with special characters that define how text or other items enclosed within the elements should be displayed. Hyperlinks are underlined or emphasized works that load to other documents or some portions of the same document.

HTML can be used to display any type of document on the host computer, which can be geographically at a different location. It is a versatile language and can be used on any platform or desktop.

HTML provides tags (special codes) to make the document look attractive. HTML tags are not case-sensitive. Using graphics, fonts, different sizes, color, etc., can enhance the presentation of the document. Anything that is not a tag is part of the document itself.

Basic HTML Tags :

<!-- -->	Specifies comments
<A>…….</A>	Creates hypertext links
<B>…….</B>	Formats text as bold
<BIG>…….</BIG>	Formats text in large font.
<BODY>…</BODY>	Contains all tags and text in the HTML document
<CENTER>...</CENTER>	Creates text
<DD>…</DD>	Definition of a term
<DL>...</DL>	Creates definition list
<FONT>…</FONT>	Formats text with a particular font
<FORM>...</FORM>	Encloses a fill-out form
<FRAME>...</FRAME>	Defines a particular frame in a set of frames
<H#>…</H#>	Creates headings of different levels
<HEAD>...</HEAD>	Contains tags that specify information about a document

<HR>...</HR>	Creates a horizontal rule
<HTML>…</HTML>	Contains all other HTML tags
<META>...</META>	Provides meta-information about a document
<SCRIPT>…</SCRIPT>	Contains client-side or server-side script
<TABLE>…</TABLE>	Creates a table
<TD>…</TD>	Indicates table data in a table
<TR>…</TR>	Designates a table row
<TH>…</TH>	Creates a heading in a table

ADVANTAGES

- A HTML document is small and hence easy to send over the net. It is small because it does not include formatted information.
- HTML is platform independent.
- HTML tags are not case-sensitive.

Java Database Connectivity

What Is JDBC?

JDBC is a Java API for executing SQL statements. (As a point of interest, JDBC is a trademarked name and is not an acronym; nevertheless, JDBC is often thought of as standing for Java Database Connectivity. It consists of a set of classes and interfaces written in the Java programming language. JDBC provides a standard API for tool/database developers and makes it possible to write database applications using a pure Java API.

Using JDBC, it is easy to send SQL statements to virtually any relational database. One can write a single program using the JDBC API, and the program will be able to send SQL statements to the appropriate database. The combinations of Java and JDBC lets a programmer write it once and run it anywhere.

What Does JDBC Do?

Simply put, JDBC makes it possible to do three things:

- Establish a connection with a database
- Send SQL statements
- Process the results.

JDBC versus ODBC and other APIs

At this point, Microsoft's ODBC (Open Database Connectivity) API is that probably the most widely used programming interface for accessing relational databases. It offers the ability to connect to almost all databases on almost all platforms.

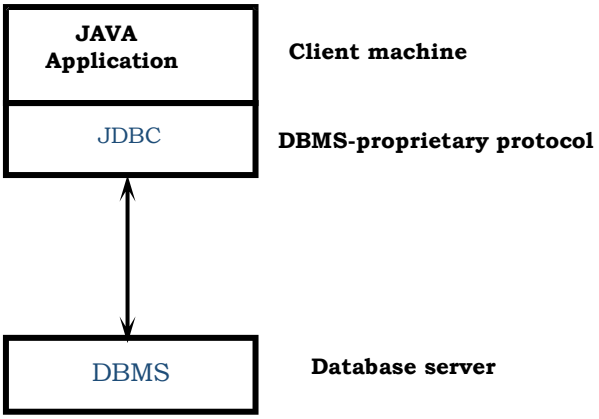
So why not just use ODBC from Java? The answer is that you can use ODBC from Java, but this is best done with the help of JDBC in the form of the JDBC-ODBC Bridge, which we will cover shortly. The question now becomes "Why do you need JDBC?" There are several answers to this question:

1. ODBC is not appropriate for direct use from Java because it uses a C interface. Calls from Java to native C code have a number of drawbacks in the security, implementation, robustness, and automatic portability of applications.
2. A literal translation of the ODBC C API into a Java API would not be desirable. For example, Java has no pointers, and ODBC makes copious use of them, including the notoriously error-prone generic pointer "void *". You can think of JDBC as ODBC translated into an object-oriented interface that is natural for Java programmers.
3. ODBC is hard to learn. It mixes simple and advanced features together, and it has complex options even for simple queries. JDBC, on the other hand, was designed to keep simple things simple while allowing more advanced capabilities where required.

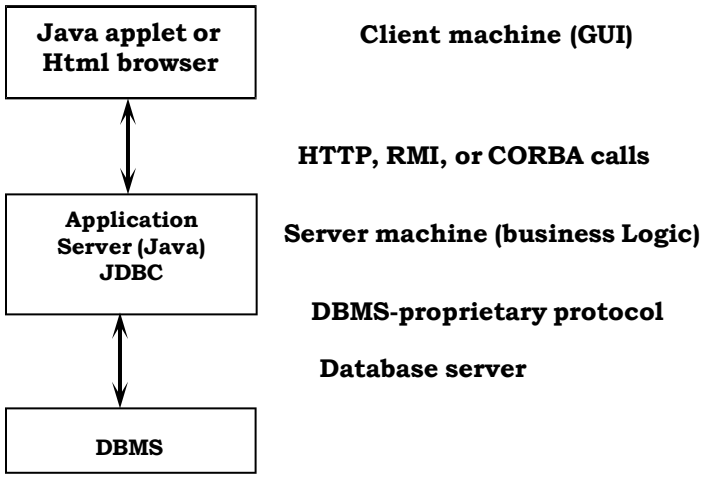
4. A Java API like JDBC is needed in order to enable a "pure Java" solution. When ODBC is used, the ODBC driver manager and drivers must be manually installed on every client machine. When the JDBC driver is written completely in Java, however, JDBC code is automatically installable, portable, and secure on all Java platforms from network computers to mainframes.

TWO – TIER AND THREE – TIER MODELS

The JDBC API supports both two-tier and three-tier models for database access. In the two-tier model, a Java applet or application talks directly to the database. This requires a JDBC driver that can communicate with the particular database management system being accessed. A user's SQL statements are delivered to the database, and the results of those statements are sent back to the user. The database may be located on another machine to which the user is connected via a network. This is referred to as a client/server configuration, with the user's machine as the client, and the machine housing the database as the server. The network can be an Intranet, which, for example, connects employees within a corporation, or it can be the Internet.



In the three-tier model, commands are sent to a "middle tier" of services, which then send SQL statements to the database. The database processes the SQL statements and sends the results back to the middle tier, which then sends them to the user. MIS directors find the three-tier model very attractive because the middle tier makes it possible to maintain



control over access and the kinds of updates that can be made to corporate data. Another advantage is that when there is a middle tier, the user can employ an easy-to-use higher-level API which is translated by the middle tier into the appropriate low-level calls. Finally, in many cases the three-tier architecture can provide performance advantages.

Until now the middle tier has typically been written in languages such as C or C++, which offer fast performance. However, with the introduction of optimizing compilers that translate Java byte code into efficient machine-specific code, it is becoming practical to implement the middle tier in Java. This is a big plus, making it possible to take advantage of Java's robustness, multithreading, and security features. JDBC is important to allow database access from a Java middle tier.

JDBC DRIVER TYPES

The JDBC drivers that we are aware of at this time fit into one of four categories:

- JDBC-ODBC bridge plus ODBC driver
- Native-API partly-Java driver
- JDBC-Net pure Java driver

- Native-protocol pure Java driver

JDBC-ODBC BRIDGE

If possible, use a Pure Java JDBC driver instead of the Bridge and an ODBC driver. This completely eliminates the client configuration required by ODBC. It also eliminates the potential that the Java VM could be corrupted by an error in the native code brought in by the Bridge (that is, the Bridge native library, the ODBC driver manager library, the ODBC driver library, and the database client library).

What Is the JDBC- ODBC Bridge?

The JDBC-ODBC Bridge is a JDBC driver, which implements JDBC operations by translating them into ODBC operations. To ODBC it appears as a normal application program. The Bridge implements JDBC for any database for which an ODBC driver is available. The Bridge is implemented as the sun.jdbc.odbc Java package and contains a native library used to access ODBC. The Bridge is a joint development of Intersolv and JavaSoft.

Java Server Pages (JSP)

Java server Pages is a simple, yet powerful technology for creating and maintaining dynamic-content web pages. Based on the Java programming language, Java Server Pages offers proven portability, open standards, and a mature re-usable component model .The Java Server Pages architecture enables the separation of content generation from content presentation. This separation not eases maintenance headaches, it also allows web team members to focus on their areas of expertise. Now, web page designer can concentrate on layout, and web application designers on programming, with minimal concern about impacting each other’s work.

FEATURES OF JSP

Portability:

Java Server Pages files can be run on any web server or web-enabled application server that provides support for them. Dubbed the JSP engine, this support involves recognition, translation, and management of the Java Server Page lifecycle and its interaction components.

Components

It was mentioned earlier that the Java Server Pages architecture can include reusable Java components. The architecture also allows for the embedding of a scripting language directly into the Java Server Pages file. The components current supported include Java Beans, and Servlets.

Processing

A Java Server Pages file is essentially an HTML document with JSP scripting or tags. The Java Server Pages file has a JSP extension to the server as a Java Server Pages file. Before the page is served, the Java Server Pages syntax is parsed and processed into a Servlet on the server side. The Servlet that is generated outputs real content in straight HTML for responding to the client.

Access Models:

A Java Server Pages file may be accessed in at least two different ways. A client's request comes directly into a Java Server Page. In this scenario, suppose the page accesses reusable Java Bean components that perform particular well-defined computations like accessing a database. The result of the Beans computations, called result sets is stored within the Bean as properties. The page uses such Beans to generate dynamic content and present it back to the client.

In both of the above cases, the page could also contain any valid Java code. Java Server

Pages architecture encourages separation of content from presentation.

Steps in the execution of a JSP Application:

1. The client sends a request to the web server for a JSP file by giving the name of the JSP file within the form tag of a HTML page.

2. This request is transferred to the JavaWebServer. At the server side JavaWebServer receives the request and if it is a request for a jsp file server gives this request to the JSP engine.

3. JSP engine is program which can understands the tags of the jsp and then it converts those tags into a Servlet program and it is stored at the server side. This Servlet is loaded in the memory and then it is executed and the result is given back to the JavaWebServer and then it is transferred back to the result is given back to the JavaWebServer and then it is transferred back to the client.

JDBC CONNECTIVITY

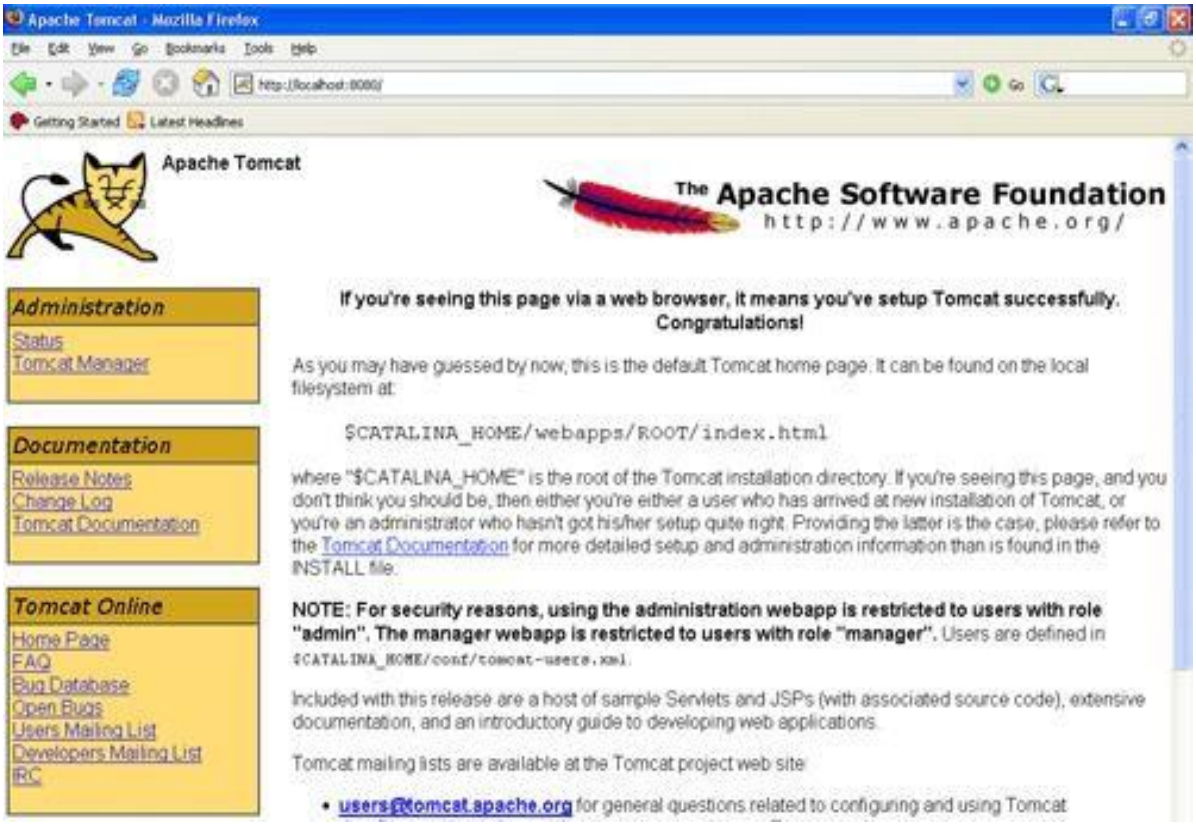
The JDBC provides database-independent connectivity between the J2EE platform and a wide range of tabular data sources. JDBC technology allows an Application Component Provider to:

- Perform connection and authentication to a database server
- Manager transactions
- Move SQL statements to a database engine for preprocessing and execution
- Execute stored procedures
- Inspect and modify the results from Select statements.

TOMCAT 6.0 WEB BROWSER

Tomcat is an open source web server developed by Apache Group. Apache Tomcat is the servlet container that is used in the official Reference Implementation for the Java Servlet and Java Server Pages technologies. The Java Servlet and Java Server Pages specifications are developed by Sun under the Java Community Process. Web Servers like Apache Tomcat support only web components while an application server supports web components as well as business components (BEAs Weblogic, is one of the popular application server).

To develop a web application with jsp/servlet install any web server like JRun, Tomcat etc to run your application.



4.2 SCREENSHOTS

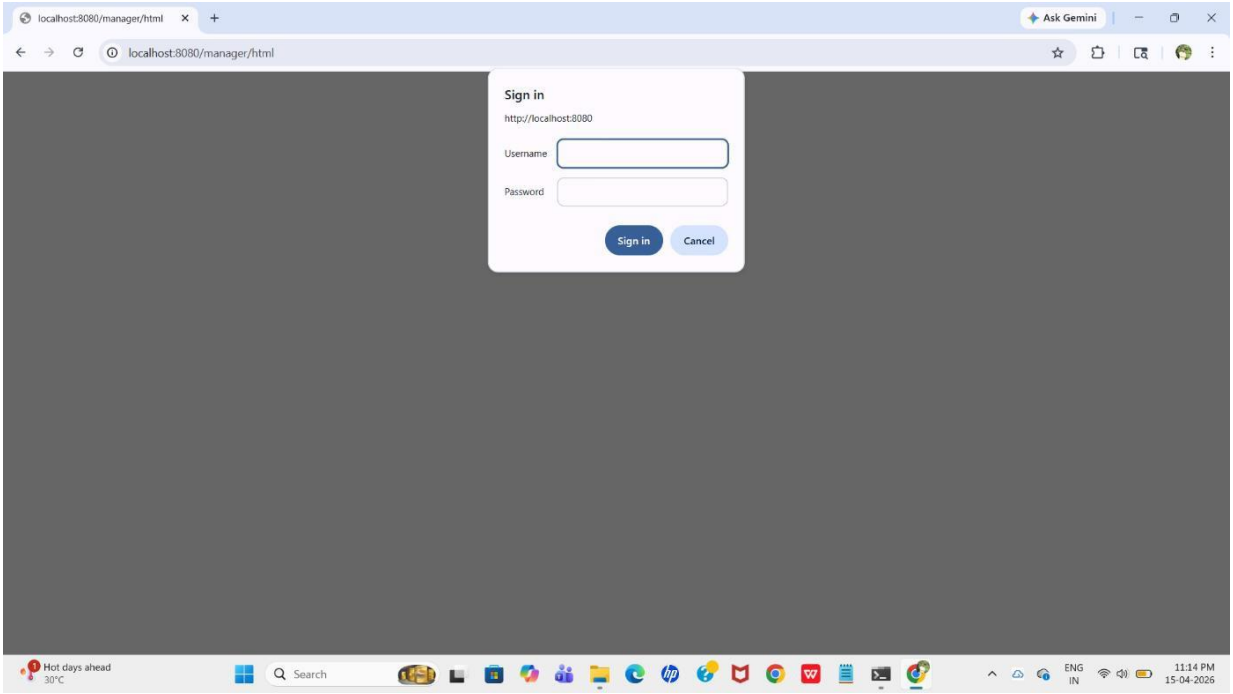


Fig1: Logging in into the **LOCAL HOST**

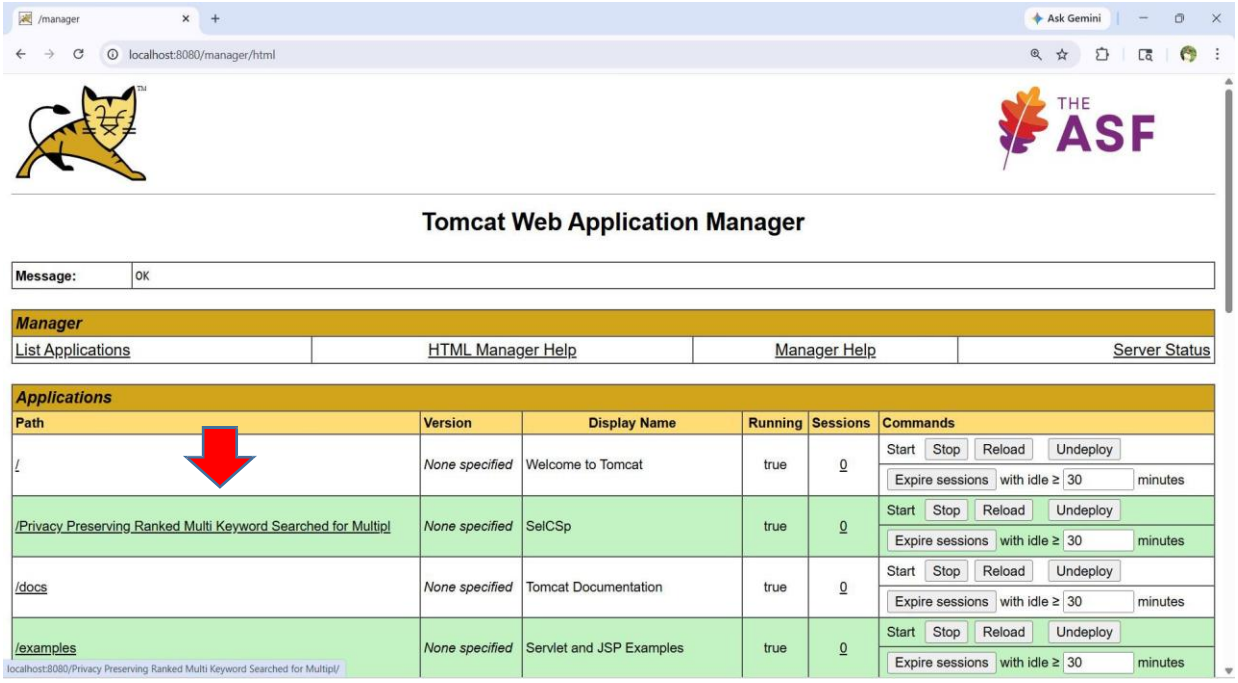


Fig2 : Tomcat Web Application Manager Displaying the Applications

- Click on the **Privacy Preserved Ranked Multi Keyword Searched for Multipl** to open the project

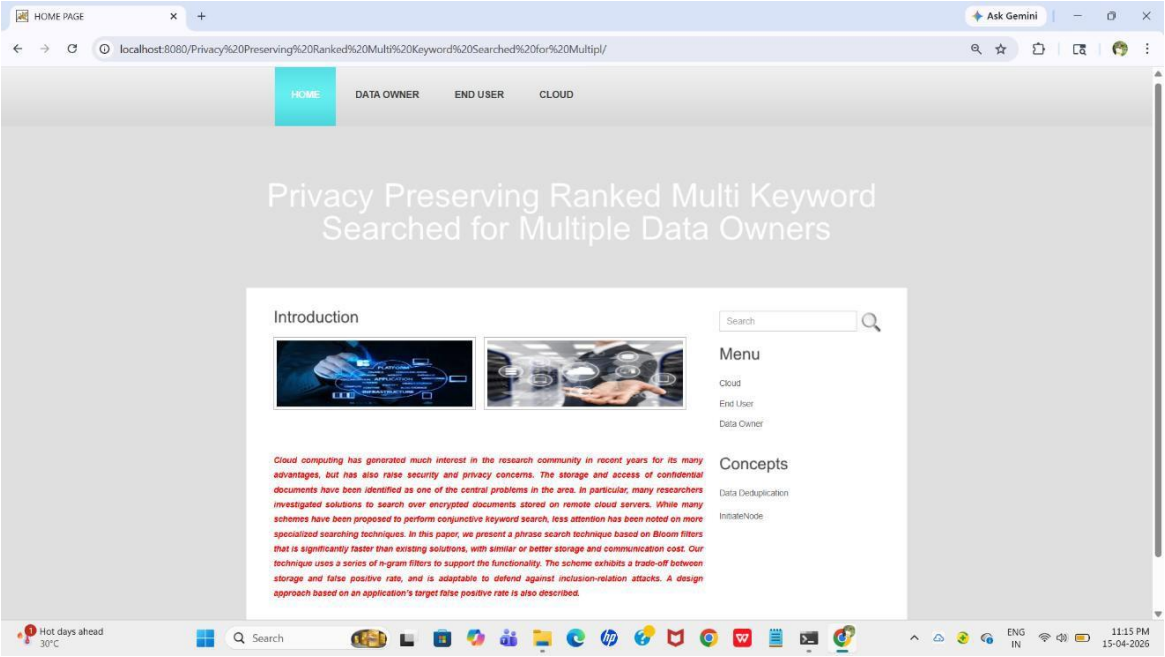


Fig3 : Home Page

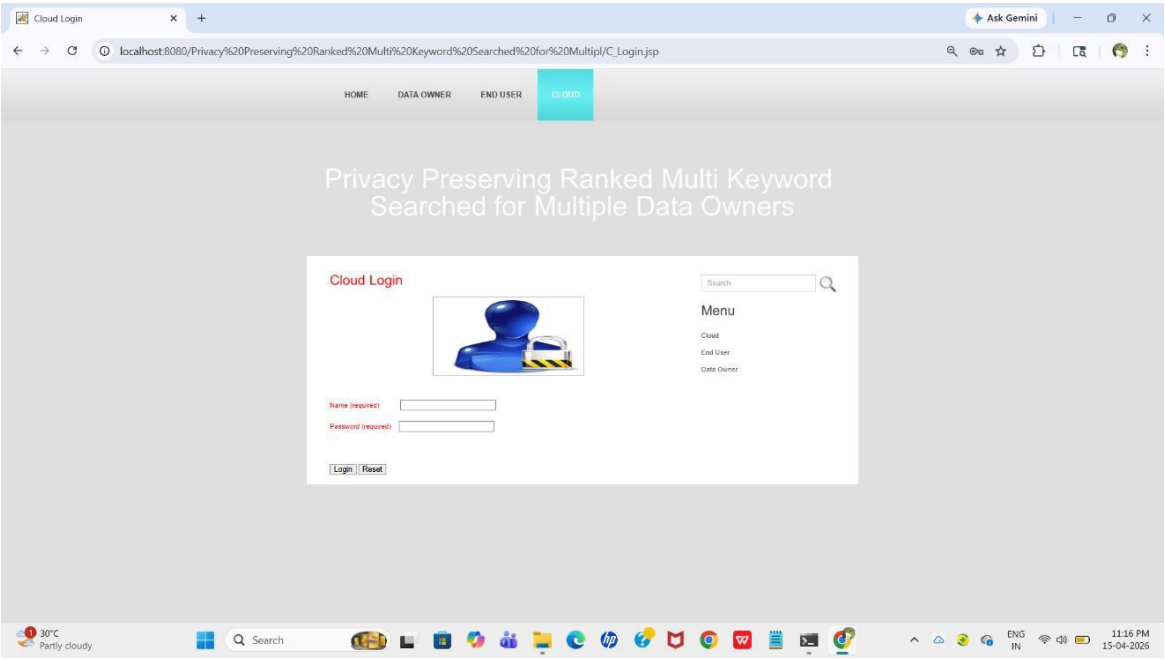


Fig4 : Cloud Admin Login Page

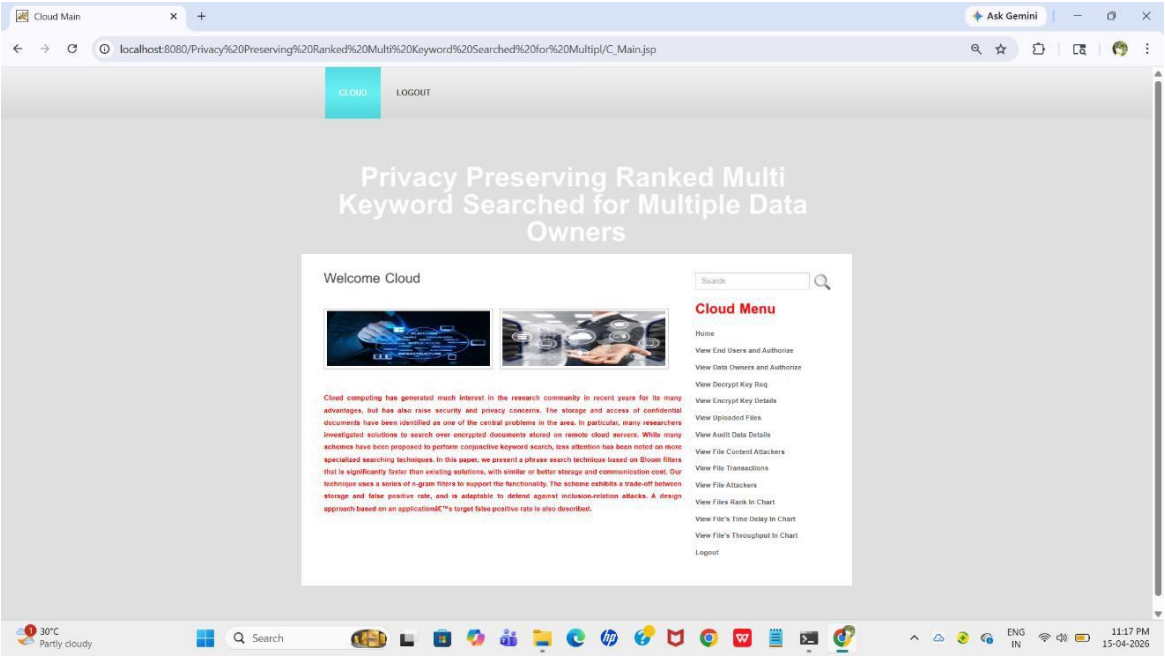


Fig5: Cloud Admin Dashboard page

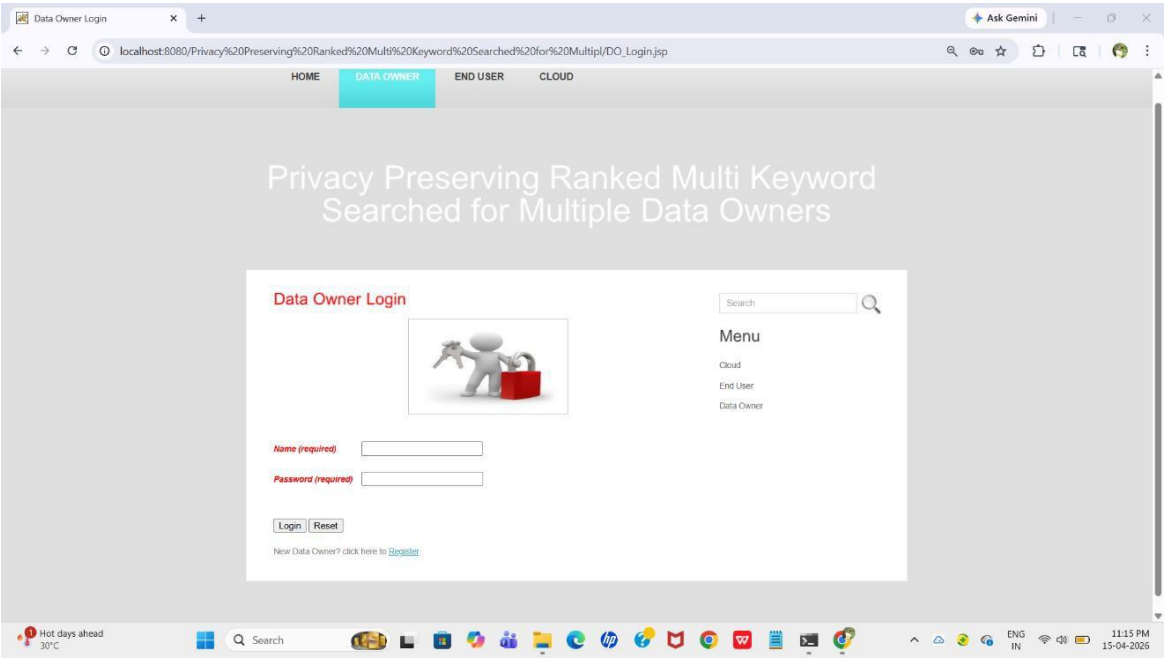


Fig6 : Data Owner Login Page

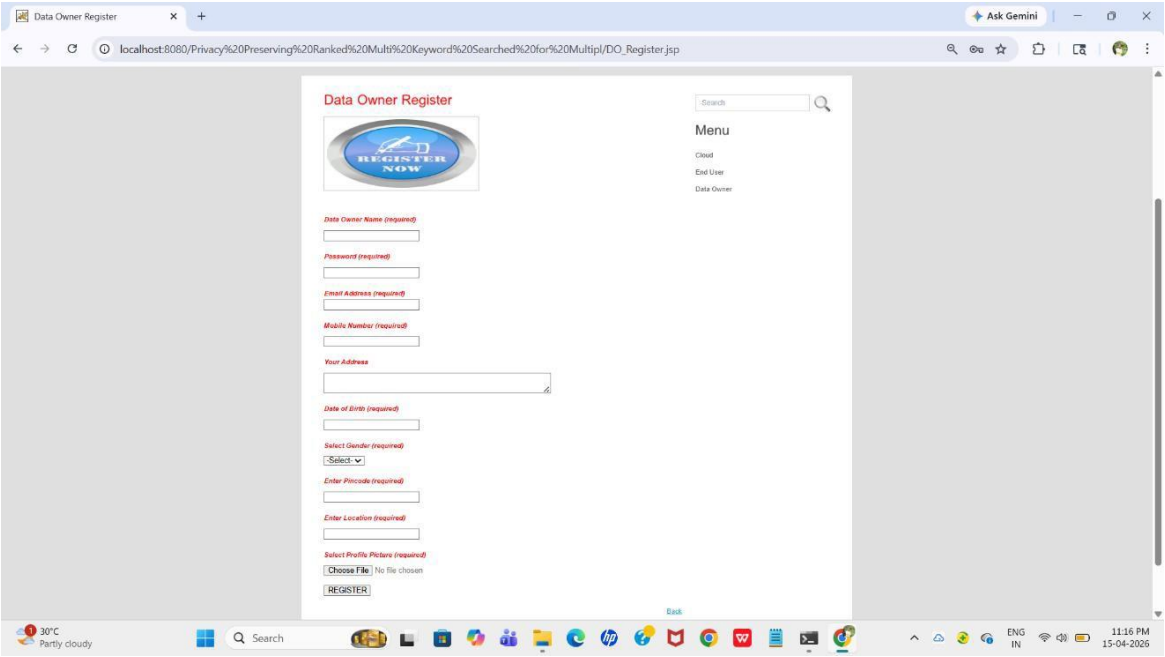


Fig7 : Data Owner Registration page

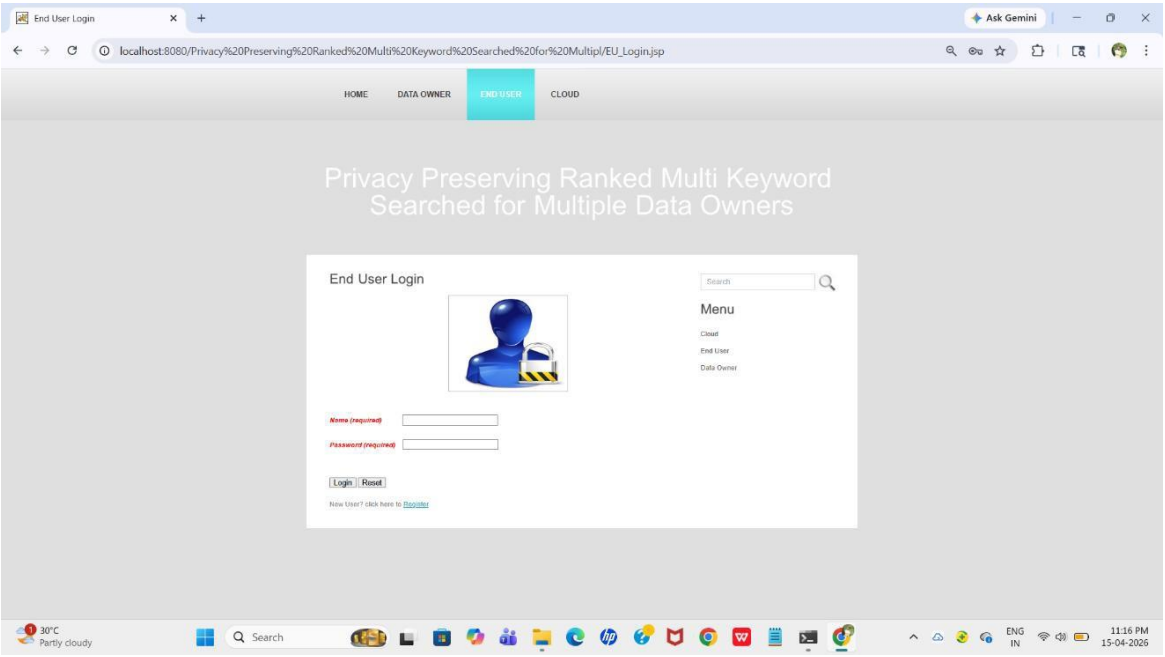


Fig8 : End User Login

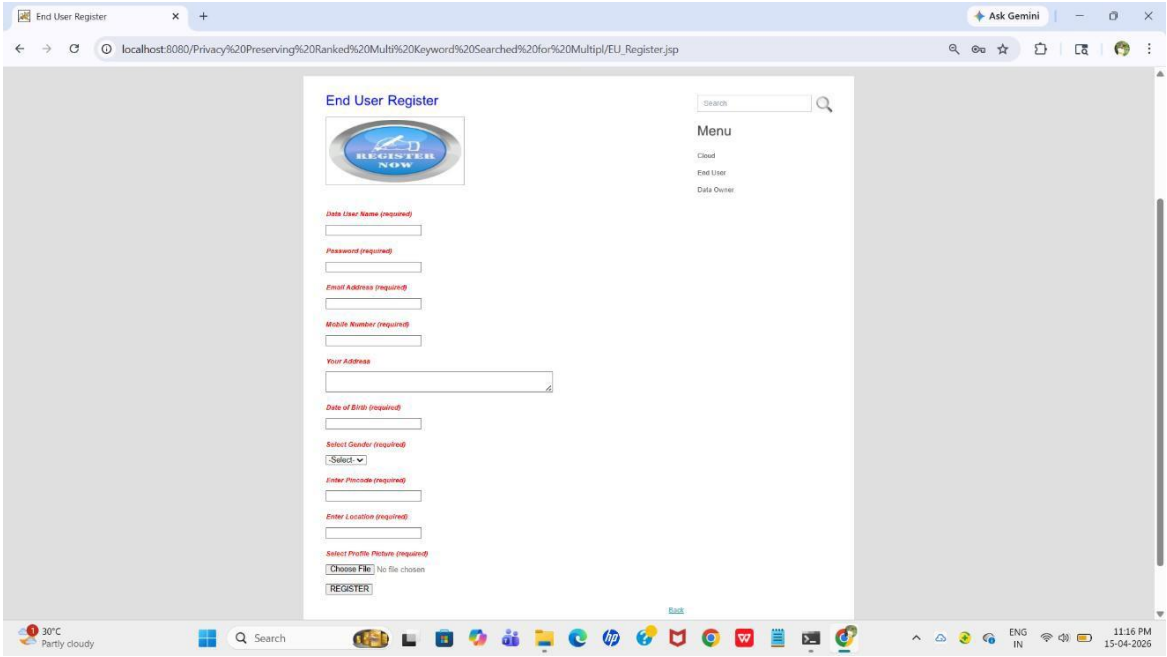


Fig9 : End User Registration

CHAPTER – 5

CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

In this paper, the proposed system for privacy-preserving ranked multi-keyword search in a multi-owner cloud environment effectively addresses the major limitations of existing searchable encryption techniques. By introducing a secure framework that supports multiple independent data owners without requiring shared keys, it ensures both scalability and data privacy. The integration of verifiability through message authentication codes guarantees the correctness and completeness of search results, while the accountability mechanism enables tracking of user activities to prevent misuse. Additionally, the use of parallel processing enhances system efficiency, making it suitable for large-scale real-world applications. Overall, this approach provides a reliable, secure, and efficient solution for encrypted data search in cloud computing, meeting the growing demands of modern multi-tenant environments

5.2 Future Enhancements

In Due Course, latest technology advancements will be taken into consideration. As part of technical build-up many components of the networking system will be generic in nature so that future projects can either use or interact with this. The future holds a lot to offer to the development and refinement of this project.

CHAPTER -6

BIBILOGRAPHY

References for the Project Development were taken from the following Books and Web Sites.

Oracle

- PL/SQL Programming by Scott Urman
- SQL complete reference by Livion

JAVA Technologies

- JAVA Complete Reference
- Java Script Programming by Yehuda Shiran
- Mastering JAVA Security
- JAVA2 Networking by Pistoria
- JAVA Security by Scotl oaks
- Head First EJB Sierra Bates
- J2EE Professional by Shadab siddiqui
- JAVA server pages by Larne Pekowsley
- JAVA Server pages by Nick Todd

HTML

- HTML Black Book by Holzner

JDBC

- Java Database Programming with JDBC by Patel moss.
- Software Engineering by Roger Pressman